RoFL: Robustness of Secure Federated Learning

Hidde Lycklama*, Lukas Burkhalter*, Alexander Viand, Nicolas Küchler, Anwar Hithnawi

ETH Zurich

2023 IEEE Symposium on Security and Privacy (SP) | 978-1-6654-9336-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/SP46215.2023.10179400

***Abstract*—Even though recent years have seen many attacks exposing severe vulnerabilities in Federated Learning (FL), a holistic understanding of what enables these attacks and how they can be mitigated effectively is still lacking. In this work, we demystify the inner workings of existing (targeted) attacks. We provide new insights into why these attacks are possible and why a definitive solution to FL robustness is challenging. We show that the need for ML algorithms to memorize tail data has significant implications for FL integrity. This phenomenon has largely been studied in the context of privacy; our analysis sheds light on its implications for ML integrity. We show that certain classes of severe attacks can be mitigated effectively by enforcing constraints such as norm bounds on clients' updates. We investigate how to efficiently incorporate these constraints into secure FL protocols in the single-server setting. Based on this, we propose RoFL, a new secure FL system that extends secure aggregation with privacy-preserving input validation. Specifically, RoFL can enforce constraints such as L_2 and L_∞ bounds on high-dimensional encrypted model updates.**

1. Introduction

Recent years have seen a surge of interest in collaborative, secure machine learning (ML) paradigms. These paradigms allow machine learning models to be trained without requiring direct access to training data, thus alleviating some of the risks associated with the large-scale collection of sensitive data. Federated Learning (FL) is a prominent form of collaborative ML that has recently emerged and is already being used in practice to train models for a variety of privacy-sensitive applications [18, 19, 74, 77, 81, 82]. In FL, the training process is distributed across a set of participants who collaborate to train a joint global model through an iterative process that aggregates locally trained updates. Although these updates contain less information than the underlying training data, they can still leak sensitive information. In fact, recent efforts have shown that by observing gradients submitted by the clients, one can reconstruct sensitive data from clients' local datasets [13, 45, 95]. Secure FL systems address this issue by employing secure aggregation [15, 54, 96] to protect clients' individual gradient updates. Here, clients send encrypted updates to the server, which can recover only the aggregated joint model update (c.f. Fig. 1). While FL provides many privacy benefits, it introduces new ML robustness issues and exacerbates existing ones. Since FL systems are open to a multitude of participants, any of whom could be compromised, the training algorithm becomes susceptible to contributions from compromised clients. In addition to manipulating the joint model by poisoning their local training data, malicious clients can also directly submit manipulated model updates. The possibility of such *model poisoning* attacks poses a unique challenge for FL systems. A flurry of work, starting shortly after the introduction of FL, has exposed the severity of both kinds of attacks in FL [38, 93]. We have seen two forms of attack goals: (i) untargeted attacks, which can easily prevent or slow learning [11, 94]. While undesirable, these predominately impact availability and are eventually detected as the server can observe that the model accuracy is not improving. Recent work by [80] studied this type of attacks and their mitigation in depth. The second form of attacks is (ii) targeted attacks [7, 9, 90, 92]. In targeted attacks, the attacker aims to integrate a backdoor into the model through specific inputs and thus cause the global model to misbehave. These pose risk to model integrity and, thus, to the robustness of FL. These have been studied to a lesser extent, and we aim to fill this gap in this paper.

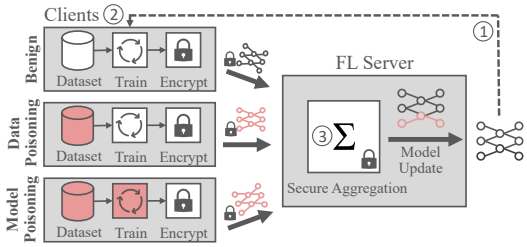


Figure 1: Secure FL pipeline with malicious clients.

A variety of defenses has been proposed against both targeted and untargeted attacks. These defenses rely primarily on anomaly detection [43, 83] or byzantine-robust estimators [11, 27, 67, 73, 94] introducing non-linear aggregation rules with audits to filter malicious updates. However, these existing solutions become prohibitively expensive when transferred to the *secure* setting, as they would require general-purpose secure computation techniques (e.g., generic secure multi-party computation (MPC) between all clients) to express them. Hence, there is a need for practical defenses that are compatible with existing secure aggregation protocols used in FL systems. Prior work has identified enforcing norm bounds on individual client updates as a computationally simple yet promising defense in practical FL setups [66, 80, 86]. This makes them a prime candidate for lifting to the secure setting because they can be expressed without requiring general-purpose MPC. Norm bounds, while not a panacea, have already been shown to prevent untargeted poisoning attacks in real-world adversarial scenarios [80]. However, to what extent norm bounds

* *These authors contributed equally to this work.*

DOI 10.1109/SP46215.2023.00189

can be effective against targeted attacks is unclear. Recent work has come to apparently conflicting conclusions in this regard [7, 90]. Reconciling these observations and working towards a remedy for the underlying issues requires a deeper understanding of the inner workings of these attacks and the factors that enable them. Our work is the first to analyze robustness for single-server secure FL more holistically and, as a result, reconciles this apparent contradiction.

We conclude that, while they have clear limits, norm bounds would indeed be an attractive robustness solution. However, this hinges on them being efficiently realizable in the secure setting. Therefore, we investigate how to efficiently incorporate such constraints into secure FL protocols in the single-server setting. Based on this, we propose a new secure FL system that extends secure aggregation with privacy-preserving input validation. Specifically, our system can enforce constraints such as L_2 and L_∞ bounds on high-dimensional encrypted model updates. As a result, this paper provides two contributions:

① Understanding FL Robustness. Even though recent years have seen many targeted poisoning attacks exposing severe vulnerabilities in FL, a holistic understanding of what enables these attacks and how they can be mitigated effectively is still lacking. To improve the understanding of FL robustness, we demystify the inner workings of existing attacks. We empirically analyze why these attacks are possible and why a definitive solution to FL robustness is challenging. We show that although some attacks are still possible under a norm bound, enforcing norm bounds significantly reduces the attack surface of secure FL. In particular, norm bounds are effective in preventing a class of highly practical attacks in which an attacker can completely replace the model by controlling just a single client. We also show that the need for ML algorithms to memorize tail subpopulations has significant implications for ML integrity. This phenomenon has largely been studied in the context of privacy [39, 40]; in our analysis, we shed light on its implications for ML robustness. For attacks exploiting model memorization, we show that norm bounds are of limited effectiveness. Nevertheless, while norm bounds are not sufficient to prevent all attacks, they increase robustness immensely in practical settings. We make our framework for analyzing FL robustness available online [1] and we hope that it will assist practitioners in better assessing the implications and risks of open FL systems.

② Secure Aggregation with Input Validation. We present RoFL, a secure FL system that enables constraints to be expressed and enforced on high-dimensional encrypted model updates to defend against attacks by malicious participants. RoFL augments existing secure FL systems [8] with zero-knowledge proofs that allow the server to enforce and verify constraints on client updates, e.g., norm bounds. In this work, we tackle the *single-server secure FL* setting proposed in Bell et al. [8], which poses unique compatibility and scalability challenges. Assuming a setting with multiple non-colluding servers (e.g., Prio [29]) enables more efficient approaches, but the required trust assumptions are hard to materialize in practice. Therefore, we focus on the (de-facto standard) single-server setting, which allows RoFL to be compatible with existing FL deployment scenarios. The key challenge of this setting is the *scale* of the problem domain, both in the number of clients that participate in each round and also the high dimensionality of the aggregation inputs, which correspond to the model size. Therefore, scalability is a key consideration in the design of a protocol for this setting. RoFL extends existing masking-based secure aggregation approaches with additively homomorphic commitments, which allows us to guarantee the correctness of the aggregation in the presence of malicious clients. We use efficient range proofs that operate directly on the additively homomorphic commitments to realize norm bound checks. We realize them using a discrete-log based zero-knowledge proof system (Bulletproofs [20]) that offers proof sizes that are logarithmic in the number of range proofs and also support batched verification, allowing our protocol to scale efficiently to large model sizes. Additionally, we introduce several optimizations at the ML layer that allow us to reduce the number of cryptographic checks needed. We implement and evaluate RoFL, showing that it scales to the model sizes used in real-world FL deployments. We make RoFL's prototype available online [2].

2. Analysis of FL Robustness

FL Background. In FL, the server coordinates the training of a model $f_{\mathbf{w}_G}$ with weights $\mathbf{w}_G \in \mathbb{R}^\ell$, where ℓ is the number of parameters, based on the datasets D_i for $i \in \{1...n\}$ held locally by a set of clients N with size $n = |N|$. At each round t of the iterative learning process, the server selects a subset of m clients and broadcasts the current global model $\mathbf{w}_G^t$ to them. Each selected client fine-tunes the model on their local training data D_i using a training algorithm such as Stochastic Gradient Descent (SGD), producing a local model $\mathbf{w}_i^{t+1}$. Each client then sends its update $\triangle\mathbf{w}_i^{t+1} \coloneqq \mathbf{w}_i^{t+1} - \mathbf{w}_G^t$ to the server. The server aggregates the updates using an aggregation function F, i.e., $\triangle\mathbf{w}_{\text{agg}}^{t+1} = F(\mathbf{w}_{i\in[m]}^{t+1})$. For FedAvg [15, 63], F_{avg} is defined as a weighted aggregation of updates. The server incorporates the aggregated updates into the next global model, i.e., $\mathbf{w}_G^{t+1} = \mathbf{w}_G^t + \eta \triangle \mathbf{w}_{\text{agg}}^{t+1}$. This process repeats until the server ends the training process.

What makes FL robustness challenging? FL is inherently open to a multitude of participants, often in the orders of hundreds of thousands. Consequently, training algorithms in such settings can be susceptible to contributions from compromised clients. Besides being susceptible to the known security and privacy vulnerabilities encountered in typical ML settings [10, 28, 53, 60], FL exposes new attack surfaces. Moreover, several FL characteristics elevate the impact of attacks known in the centralized setting, such as backdoor attacks through data poisoning [7, 9, 90, 92]. These unique challenges are attributed to three characteristics: *(i) Open Nature:* In FL, the training process is open and often a

1. https://github.com/pps-lab/fl-analysis

2. https://github.com/pps-lab/rofl-project-code

large number of participants is involved, among whom any participant can act maliciously. *(ii) Attackers' Capabilities:* In conventional centralized ML settings, an adversary can instigate targeted attacks only by data poisoning. In the FL setting, however, an attacker can do so by directly manipulating model updates to enhance their impact, allowing more effective attacks (i.e., model poisoning). *(iii) Active Attackers:* An attacker that controls a subset of clients can observe and influence the model behavior over multiple training rounds, which can allow for stronger forms of adaptive attacks.

Analysis Methodology. In this section, we extensively investigate FL robustness in the presence of malicious participants and answer to what extent norm bounding can effectively defend against malicious attacks. We primarily focus on *targeted* attacks where the adversary compromises the global model's integrity by causing it to misclassify a target set of samples. Recent work [80] has already addressed untargeted attacks in-depth and showed the high effectiveness of norm bounding against untargeted attacks in practice. Therefore, we refer to [80] for the analysis of untargeted attacks and only provide a few additional insights not covered in the existing work in Appendix A.

Initially, proposed targeted attacks [7, 9] for FL relied heavily on update scaling to succeed. Intuitively, attacks relying on scaling can be mitigated effectively with norm bounds [80, 86] that limit participants' contributions to the model to prevent a single participant from overpowering the aggregation process. More recently, however, attacks have been shown to be successful even in the presence of norm bounds. Consequently, this has called the ability of norm bounds to mitigate targeted attacks into question. Understanding to what extent and why norm bounds are (in)effective requires a deeper understanding of the factors that enable them.

In this work, we study the different attacks from the perspective of the targets they attack and relate this to insights into the nature of the input data distribution of common deep learning tasks. In particular, it has been widely observed that modern datasets of natural images and text data follow long-tailed distributions, which can be seen as mixtures of *subpopulations* [39, 40, 89, 99]. The long-tailed nature implies that a significant fraction of the samples in the dataset belong to rare subpopulations, such as the images of cars from unusual angles in Fig. 2. We use this to categorize the inputs that are targeted by attacks and then study attacks on two types of subpopulations representing opposing ends of the input distribution. The first type is *prototypical* targets, i.e., subpopulations of samples that frequently occur in the dataset. The second type is *tail* targets, i.e., subpopulations that sit on the long tail of the distribution and rarely occur in benign clients' datasets. We show how attack effectiveness varies dramatically depending on the type of subpopulation targeted. We find that analyzing robustness issues through the lens of the data distribution is crucial, as certain attacks exploit maliciously chosen input data (data poisoning) rather than deviations from the protocol [87, 90].

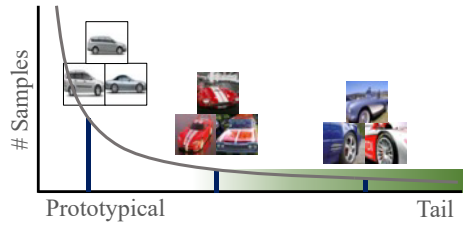


Figure 2: Modern datasets can be modeled as a long-tailed mixture of subpopulations, with the tail consisting of subpopulations of rare samples [89, 99].

2.1. Attack Strategies

We focus on targeted attacks that aims to violate the integrity of the global model [7, 9, 86, 90] by causing it to misclassify a subset of unmodified data samples while maintaining high accuracy on the main task. Specifically, a training set samples $\hat{D}$ that represent the target inputs are classified as a target class $\hat{t}$ rather than their ground-truth class.

Adversary Capabilities. The adversary can compromise a subset of client devices and adapt their training process to influence the global model $\mathbf{w}_G^{t+1}$ by submitting malicious updates to the server. We consider two adversary models, a model-poisoning adversary and a data-poisoning adversary (Fig. 1). The model-poisoning adversary can submit arbitrarily generated updates, while the data-poisoning adversary is limited to changing compromised clients' training data. We use the same realistic model poisoning threat model identified in [80] (Sec III-C2) where the adversary can only compromise a small fraction of clients ($< 5\%$ in a round). This is a reasonable assumption because a typical FL setup consists of hundreds of thousands of clients, and only a small fraction of them is selected in each training round. When a norm-bound defense is in place, the server can enforce an upper limit B on a p-norm of each client update such that $||\Delta\hat{\mathbf{w}}||_p \leq B$ [86, 90]. We assume that the attacker and clients are aware of this bound and that honest clients clip their updates to stay within the bound.

Data poisoning (DP). The adversary has access to the local dataset of compromised clients but cannot alter the training algorithm. We consider the widely used label flipping strategy [90] where the compromised clients' dataset contains a combination of benign samples D_i and target samples $\hat{D} = \{(x, \hat{t})\}$ with the labels flipped to the target class $\hat{t}$.

Model Poisoning (MP). In model poisoning, the adversary exploits its complete control over the training process to craft an arbitrary malicious update $\Delta\hat{\mathbf{w}}$. Specifically, in a model-replacement attack [7, 86, 90], the adversary injects the desired attack target into their local model by fine-tuning their local model on both the benign and backdoor samples $D_i \cup \hat{D}$ using SGD, and then applies scaling to amplify the update to survive aggregation. This exploits the inherent vulnerability of linear aggregation functions to outliers, where the update of even a single client can influence the output of F_{avg} arbitrarily [11]. If a norm bound is present, the adversary adapts their strategy to the norm constraint.

We consider three state-of-the-art adaptive attacks for the norm bound:

I) Projected Gradient Descent [86, 90](MP-PD). The adversary performs fine-tuning of the local update with SGD, but projects the update $\Delta\hat{\mathbf{w}}_i$ onto a constraint set after every iteration. This constraint set is defined such that the update satisfies the norm bound after scaling, i.e., $||\Delta\hat{\mathbf{w}}_i||_p \leq \frac{B}{\gamma}$, after being scaled by the scaling factor $\gamma = \frac{B}{||\Delta\hat{\mathbf{w}}_i||_p}$.

II) Neurotoxin [98] (MP-NT). This adaptive attack improves the durability of backdoors by poisoning specific weights that benign clients are unlikely to update. The adversary first determines $M = top_k(\mathbf{w}_G^t, \mathbf{w}_G^{t-1})$, the top-$k\%$ weights of $\mathbf{w}_G$ that are infrequently updated by comparing the current global model with the model from the previous round. The adversary then uses PGD to project their update on the coordinate-wise constraint $\Delta\hat{\mathbf{w}}_i \cup M = 0$.

III) Anticipate [91] (MP-AT). This adaptive attack is based on the idea that the optimal malicious update $\Delta\hat{\mathbf{w}}_i$ should take into account the effect of the aggregation and further training rounds. The attack tries to simulate the effect of the honest clients on the malicious update, including how future rounds will affect it. In order to compute the malicious update, the attack simulates multiple FL rounds locally in each optimization step and back-propagates through these simulated future updates.

2.2. Experimental Setup

We provide a brief overview of the experimental setup and refer the reader to Appendix §C for more details.

Analysis Tasks. Our first task (**FMN**) is a digit classification task on the Federated-MNIST dataset using the LeNet5 [58] architecture following [86, 90]. The second task (**C10**) is an image classification on the CIFAR-10 dataset using the ResNet-20 [51] CNN. We divide the dataset among 3383/100 (**FMN/C10**) clients in a non-IID fashion and select 30/40 clients in each round.

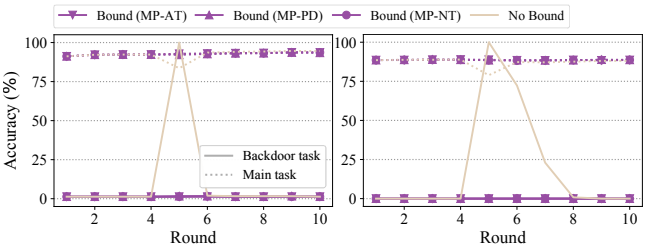
Attack Tasks. The attacks targeting prototypical inputs are: (**FMN-P**), which classifies images containing '7' as '1' instead [86], and (**C10-P**), which classifies images of green cars as birds [7]. The tail backdoor attacks target the two following inputs: (**FMN-T**), which classifies European-style '7' as '1' [90], and (**C10-T**), which misclassifies images of Southwest airline planes (which are not present in the original dataset) as trucks [90].

Client Selection. For both tasks, a constant number of clients m out of n total clients is selected in every round. We assume that the attacker controls a fraction $\alpha \in [0, 1]$ of the selected clients either for a single or multiple rounds.

Metrics. We consider two metrics: main task accuracy and backdoor task accuracy. Main task accuracy denotes performance of the model on the benign objective, that is, the model's accuracy on the benign test set. The backdoor task accuracy reports the performance of the model on the malicious objective, which we define as the model's accuracy on a subset of the targeted backdoor images.

2.3. Attacks on Prototypical Targets

We first examine attacks on prototypical subpopulations.



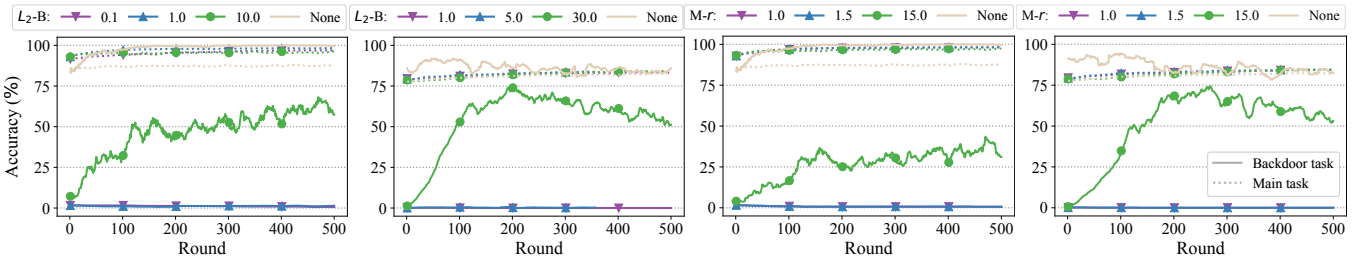
(a) **FMN** (b) **C10**

Figure 3: Single-shot model-replacement attack [7] at round 5. The attacker can inject the backdoor in a single round by dominating the aggregation with scaling. (L_2-norm bound with three adaptive attacks, **FMN**: 4.0, **C10**: 5.0)

Single-shot Attack. Even an adversary controlling a single client (**FMN**: $\alpha = 3.3\%$, **C10**: $\alpha = 2.5\%$) that is selected in a single round can inject a backdoor by scaling its update by a factor of 30 (**FMN**) and 100 (**C10**) respectively, as Figure 3 shows. The backdoor is injected in the fifth round, after the model has achieved high accuracy on the main task, so that updates from benign clients only marginally change the model. The ability of an attacker to introduce backdoors targeting prototypical subpopulations is conditioned on the attacker's ability to scale its malicious contribution to overpower the benign clients' contributions. Specifically, a server that enforces an L_2-norm bound (**FMN**: 4.0, **C10**: 5.0 in Fig. 3) can prevent single-shot attacks for all adaptive attack strategies that we study, demonstrating the effectiveness of a constraint-based approach.

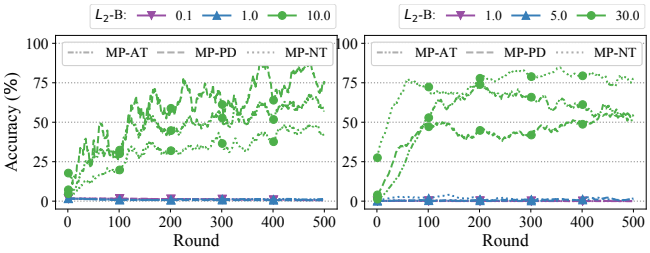
Continuous Attack. We now evaluate the effectiveness of the norm-bound defense against a continuous attacker, i.e., an attacker that controls a client participating in multiple rounds. An attacker controlling a single-client (**FMN**: $\alpha = 3.3\%$, **C10**: $\alpha = 2.5\%$) that is selected every round can maintain high accuracy on the malicious objective in the global model (**FMN**: $> 95\%$, **C10**: $> 80\%$) by continuously providing malicious updates (Fig. 4). We see that limiting the client contributions with an L_2-norm bound allows the server to reduce the attacker's success of injecting a prototypical backdoor (Figs. 4a and 4b). A norm bound of 1.0 (**FMN**) and 5.0 (**C10**) prevents the prototypical backdoor attack with hardly any classification accuracy loss on the main task. Although the attacker can send an update every round, the benign clients dominate the aggregation because the effect of scaling is limited by the bound. In Figs. 4a and 4b we only show the state-of-the-art MP-AT attack [91]. We additionally compare all three adaptive model poisoning strategies in Fig. 5. For an appropriate norm bound, attacks perform similarly with less than ten percent variation in backdoor accuracy and all three attacks fail to inject the backdoor. However, the attacks' performance diverges when the bound is too loose (10 for **FMN** and 30 for **C10**). Since they perform similarly under appropriate norm bounds, we omit the other attacks in some figures to improve readability. We refer to Appendix §A for the full results.

Norm Bound Selection. The norm bound value must be chosen carefully if it is to be effective against prototypical



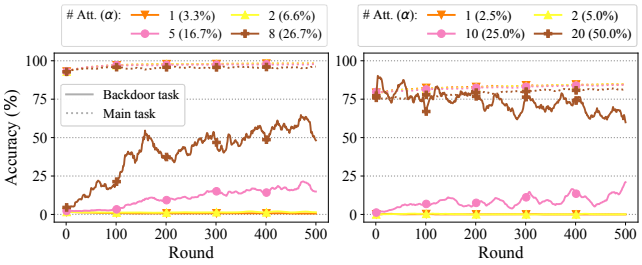
(a) **FMN-P**: Static (b) **C10-P**: Static (c) **FMN-P**: Adaptive (d) **C10-P**: Adaptive

Figure 4: Anticipate attack under various static and adaptive norm bounds. The prototypical backdoor attack can be prevented by choosing an appropriate static or adaptive median-based norm bound.



(a) **FMN-P** (b) **C10-P**

Figure 5: Comparison of adaptive attacks for various bounds.



(a) **FMN-P** (b) **C10-P**

Figure 6: Continuous MP-AT attack for % of compromised clients per round. (M-r: 1.5)

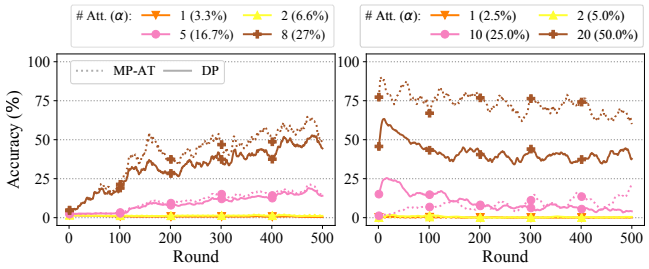
backdoor attacks without interfering with the performance of the global model. If the bound is too loose, the attacker can exploit larger scaling factors to successfully inject a backdoor. Conversely, if the server selects the bound too tightly (i.e., 0.1 for **FMN** and 1.0 for **C10**), the convergence rate of the global model decreases unnecessarily because the honest clients have to clip their updates by a large margin (Figs. 4c and 4d). Intuitively, an effective norm bound should be close to the size of benign clients' updates. However, what constitutes a suitable norm bound is not uniform across FL deployments and depends on factors such as the model architecture, training hyperparameters, and the data distribution of the clients. In addition, the size of benign clients' updates typically becomes smaller as the model converges. A bound set at the beginning would potentially give the adversary more influence in later stages of training. Hence, instead of setting a static bound, we should select the norm bound dynamically relative to the size of the benign clients' current updates.

However, estimating the average norm of benign updates is challenging due to the presence of malicious clients. Simple approaches, such as the mean or average of the client update sizes, can easily be manipulated by individual malicious outliers. Instead, we need to use a statistic that is robust to outliers, such as the median, which can tolerate up to 50% malicious inputs. Note that in the secure FL setting where the server does not have direct access to client updates, the robustness of the median also removes the need to cryptographically verify clients' reported[3] update sizes.

The median gives us a bound that is relative to the typical client update, and that adjusts over time as the model converges. However, directly using the median m as the bound would discard half the updates, drastically slowing the convergence rate. Instead, the bound is set to rm for a small multiplier r. This multiplier is a hyperparameter that must be tuned in combination with the other training hyperparameters, but is less dependent on components such as model architecture than a static norm because of its relativity to the median client update norm. Figs. 4c and 4d show that a median-based bound with $r = 1.5$ successfully prevents attacks without impacting the convergence times of the main task. This translates to absolute L_2-norm bounds in the range of $[0.24, 0.86]$ for **FMN** and $[1.96, 2.81]$ for **C10**. For the rest of this analysis, we adopt the median-based selection strategy and use $r = 1.5$ across tasks.

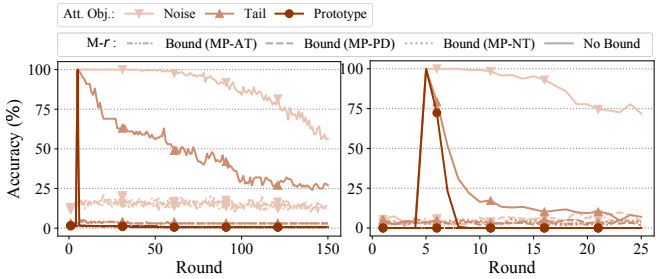
Growing Number of Compromised Clients. So far, we have considered an adversary that controls a single client in each round. In Fig. 6, we see that when the attacker controls more clients per round, the effectiveness of injecting a prototypical backdoor increases even if a norm bound is enforced. In these experiments, the strategy of the attacker is to divide the scaled update among the compromised clients so that the individual scaling factor for each malicious client becomes smaller. For **FMN**, we see that the malicious objectives' success is close to zero for $\alpha \leq 6.6\%$ but above 50% for $\alpha = 26.7\%$. Similarly, for **C10**, the accuracy of the malicious objective is low for a small number of attackers ($\alpha \leq 5.0\%$), but increases to more than 50% when $\alpha = 50\%$. We observe that scaling is less relevant for attack success if the attacker controls enough clients in each round.

3. Revealing the client update norm should be acceptable in practice because it affects privacy only marginally. However, privacy-preserving cryptographic protocols to approximate the median are available [14, 29].



(a) **FMN-P** (b) **C10-P**

Figure 7: Comparison of model poisoning and data poisoning attack strategies for % of compromised clients per round. The backdoor accuracy difference between model and data poisoning is small. (M-r: 1.5)



(a) **FMN** (b) **C10**

Figure 8: Comparison of prototypical and tail attack targets for the single-shot attack. Prototypical targets are quickly reversed by the honest clients, but tail targets stay in the model longer. (M-r: 1.5)

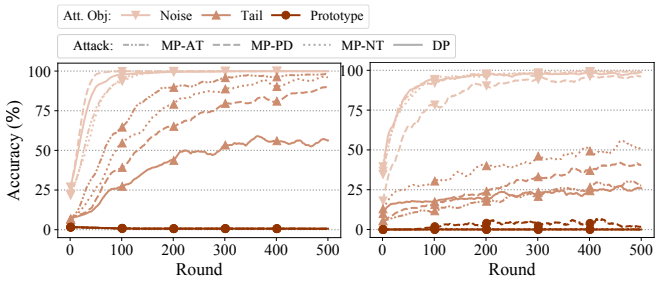
This observation is supported in Fig. 7, which shows a comparison of model poisoning (MP-AT) with data poisoning. Data poisoning shows a similar effectiveness improvement as the number of compromised clients per round increases, staying within at least 60% of the backdoor accuracy of model poisoning for both tasks. This suggests that we can use norm bounding to eliminate or reduce the advantage an attacker gains from model poisoning to the point where the effectiveness of their attack is only marginally better than honestly following the protocol and simply training on poisoned data. Of course, if an adversary can continuously influence a significant fraction of the selected clients, the definition of FL implies that their malicious information will be incorporated into the global model. However, in practical FL deployments, it is unlikely that an attacker will be able to continuously control a large fraction of the clients each round, as they are selected randomly from a much larger pool of available clients.

2.4. Attacks on Tail Targets

So far, we have discussed how attackers can embed backdoors in models by using scaling, which exploits the susceptibility inherent in the underlying linear aggregation rules used in FL. All the early targeted attacks that were proposed on FL [7, 9] relied on scaling for attack success. More recently, however, work has emerged that shows that introducing backdoors can be achieved without scaling if the attacker selects a particular subset of inputs as its attack target [90]. In the following, we provide insights into why this is possible and what aspect of the learning process these attacks exploit.

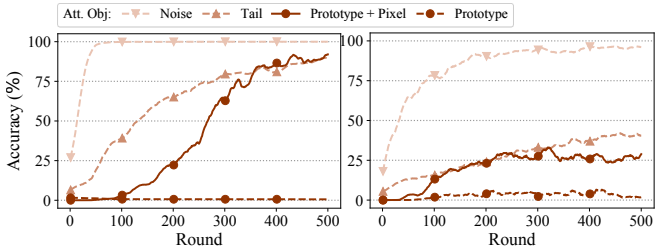
Single-shot Attack. Before we explore why these attacks are possible, we discuss some empirical observations to compare the performance of prototypical and tail attack targets for model poisoning and data poisoning attacks. In Fig. 8, we can see a comparison between tail backdoors (**FMN-T**, **C10-T**) and prototypical backdoors (**FMN-P**, **C10-P**) for a single-shot attacker. We observe that a tail backdoor remains in the global model for a very long time (at least 200 rounds for **FMN**) after its initial introduction, whereas the prototypical backdoor is almost completely eliminated only a few rounds after its introduction. This is primarily because honest clients are unlikely to submit updates that strongly influence model behavior for the subpopulation of the tail backdoor since it is less likely that the benign clients' training data contains data points from the tail of the distribution. To further highlight this phenomenon, we experiment with data samples of random (Gaussian) noise labeled as the number '2' (**FMN**) and birds (**C10**) as the backdoor target. We use these to emulate the behavior of the model on extreme tail samples. The model memorizes the noise perfectly for many rounds before being overwritten by benign clients' updates, suggesting that the farther backdoor targets are away from the data distributions of benign clients, the longer the backdoor stays in the model. Nevertheless, similar to prototypical targets, a single-shot tail attack still fails to inject the behavior successfully when a norm bound is enforced (Fig. 8).

Continuous Attack. In contrast to all other settings, a continuous attacker can slowly inject a tail backdoor even in the presence of a norm bound. However, injection is not instantaneous (c.f. single-shot tail attacks, Fig. 8) and requires the attacker to participate continuously over many rounds. Backdoor accuracy reaches 50% only after 75 rounds for **FMN** and 500 for **C10** (Fig. 9). In contrast, the prototypical backdoor attack's success remains below 10% throughout the whole training process for both tasks. This difference is because, for tail targets, the attacker requires less influence over the aggregation process to succeed than for prototypical targets, as the malicious behavior is slowly learned by the global model. This is also apparent for the noise backdoor target, where the model learns to almost perfectly memorize samples of random noise in less than 100 rounds. Data poisoning is similarly effective as model poisoning at injecting tail targets in both tasks (within 50% of the accuracy of MP) and noise targets (similar accuracy as MP). This indicates that scaling is less important for continuous tail attacks, allowing them to succeed even when the norm bound is tight. These results indicate that these tail attacks exploit the characteristics of the learning algorithm and not the aggregation rule itself. Based on our observations, we would expect previously unsuccessful poisoning attacks on prototypical targets to succeed once we artificially modify the samples with the intention making them less prototypical and more like the tail targets we studied above. To investigate this,



(a) **FMN** (b) **C10**

Figure 9: Comparison of prototypical and tail attack targets for the continuous attack under a norm bound. (M-r: 1.5)



(a) **FMN** (b) **C10**

Figure 10: Adding a pixel pattern to prototypical targets makes them behave similarly to tail targets. (M-r: 1.5)

we add an artificial trigger [50] to a set of prototypical attack targets. In Fig. 10, we show that it is indeed the case that the attack on the modified prototypical targets is successful, with an improvement in backdoor accuracy of more than 85% for **FMN** and 25% for **C10**. This is possible because artificial triggers can move samples into a new, rare, subpopulation, enabling the same attacks that work on naturally rare subpopulations.

Takeaway: Continuous attacks on tail targets *cannot* be prevented using a norm bound, because these attacks exploit the models learning capacity to embed their backdoors. Our results align with other empirical results [97] and a recent theoretical explanation [39, 40] that memorizing data at the tail of the distribution is an inherent and essential property of modern over-parameterized learning models. Machine learning memorization's relation to data privacy has been explored in depth. Research shows that this is what enables membership inference attacks [85] or attacks that extract sensitive data from the model [24, 25]. However, with the emergence of learning paradigms such as FL that are vulnerable to active attacks, memorization can also pose serious risks to robustness. Because it is memorization that allows attackers to embed tail backdoors, attackers only need to participate in the learning process with malformed data and otherwise follow the protocol honestly. Even a single attacking client, if participating over multiple rounds, is sufficient for the backdoor to be embedded persistently. Our analysis shows that norm bounds effectively reduce the attack surface of FL while highlighting the limits of the approach.

Norm bounds provide useful robustness guarantees because of the ease of performing practical high-impact single-shot attacks in standard secure FL. While it is likely that an attacker will be selected at least once, thus enabling a single-shot attack, being selected consistently over many rounds to enable a continuous attack is unlikely in a practical FL setup [80]. Achieving this persistence would require an attacker to compromise a significant portion ($> 1\%$) of clients, which corresponds to hundreds of thousands of devices in large-scale FL deployments. We thus conclude that norm bounds provide important practical robustness guarantees for FL.

2.5. Discussion

As ML is deployed in a wider range of settings, it has become clear that it must move beyond simply pursuing accuracy and start to tackle important objectives that matter in real-world deployments, such as privacy and robustness. FL, though presenting many privacy benefits, amplifies ML robustness issues by exposing the learning to an active attacker that can be present throughout the training process and fine-tune their attacks to the current state of the model. In consequence, we have seen new and powerful attacks emerge that effectively impact FL integrity. In this analysis, we show that norm bounding can provide meaningful, practical robustness guarantees for FL. Furthermore, our analysis shows that many attacks that remain possible in the presence of norm bounds are likely inherent to the learning process. For example, norm bounds are not able to prevent an attack targeting the tail of the input distribution without sabotaging the model's ability to learn less representative data points. In general, defenses against all possible attacks are likely an unattainable goal, and we should not refrain from strengthening our systems with existing solutions while we continue to expand our understanding of the underlying robustness issues of decentralized learning. This mirrors trends in the ML community, moving from a binary view of robustness to a more differentiated way of addressing adversarial behavior.

The requirement for ML algorithms to memorize in order to learn tail subpopulations' specific behavior [39] has largely been studied in the context of privacy. However, as we have shown in this analysis, it also has significant implications for ML integrity. Consequently, mechanisms that have been employed to limit memorization, such as appropriate regularization and noise addition (i.e., differential privacy) are also likely to improve ML robustness for tail targets [39]. However, these countermeasures introduce various tradeoffs with respect to the accuracy, privacy, robustness, and fairness [6] and are beyond the scope of this paper.

The appeal of norm bounds is that they exist at an intersection of effectiveness and efficiency where they prevent an extensive range of simple-to-execute yet devastating attacks in practice while remaining efficiently implementable. In addition, they do so in a robust way that does not require obfuscation (i.e., the protection still holds if the attacker knows the bound). Norm bounds increase robustness immensely in practical settings, and more extensive protections currently require sacrificing privacy, efficiency,

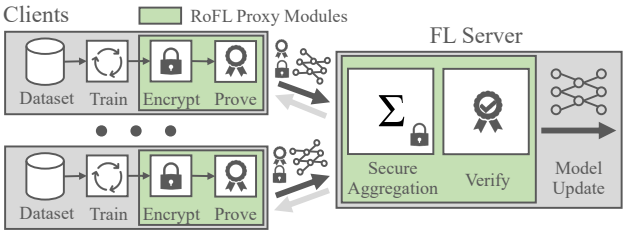


Figure 11: RoFL augments secure FL with proof and verification steps to verify constraints on private model updates.

or both [11, 27, 43, 67, 73, 83, 94]. However, the privacy-preserving nature of secure-aggregation-based FL means that enforcing norm bounds is non-trivial. Relying on clients to self-regulate is not viable in the presence of actively malicious attackers, and using generic secure computation tools would be prohibitively expensive. Therefore, we need solutions that can enforce norm bounds over (private) updates from untrusted clients with an overhead small enough to enable practical deployment. The remaining of this paper aims to address this challenge.

3. RoFL Design

In this section, we present RoFL, a new secure aggregation protocol that allows the server to verify that clients' secret inputs satisfy a predefined constraint. We start the section with a brief overview of the cryptographic blocks used in RoFL, describe our setup and threat model, and then present the details of our protocol.

3.1. Preliminaries

Commitments. A hiding non-interactive commitment scheme is a cryptographic primitive that allows an entity to commit to a chosen value v while keeping it secret, with the option of later revealing the value if needed. A commitment scheme takes a message v from a message space M and randomness r from a randomness space R, and computes a commitment c to the value v as $c = Com(v, r)$. To verify the commitment, the entity can reveal r and v, and the commitment can be checked by verifying that $c_v = Com(v, r)$. A commitment is *hiding* if it does not reveal any information about the committed value v, and is *binding* if it is infeasible to find v, r and v', r' such that $v \neq v'$ but $Com(v, r) = Com(v', r')$. The level of security offered by these properties may vary among different commitment schemes, depending on whether the properties hold against any adversary (i.e., information-theoretic security) or only against computationally bounded adversaries. In the context of this section, *binding* refers to being computationally binding, and *hiding* refers to being information-theoretically hiding, unless explicitly stated otherwise.

Zero-Knowledge Arguments of Knowledge. A zero-knowledge *argument of knowledge* is an interactive protocol that enables a prover to convince a verifier of their knowledge of a secret without revealing any information about it. For instance, a prover could demonstrate their knowledge of the discrete log x of a group element g^x and that x is within a specific range without revealing x. More formally, a zero-knowledge argument of knowledge is an interactive protocol to prove knowledge of a *witness* x for a *statement* y so that $(x, y) \in \mathcal{R}$, an NP relation that defines whether or not a given witness is valid for a statement. Interactive protocols can be converted into a non-interactive zero-knowledge proof (NIZK) [12] using the Fiat–Shamir transform [41]. For brevity, we refer to Bünz et al. [20] for formal definitions of the security properties of NIZKs. Note that a zero-knowledge protocol that only holds against a computationally bounded prover is referred to as an *argument* rather than a *proof* with respect to its soundness property. However, for the rest of this paper, we will not make this distinction unless it is relevant.

3.2. Setup and Threat Model

RoFL is designed for a standard FL deployment featuring a single server coordinating a model learning process with a large number of clients (Fig. 11). In each round of the training process, the server randomly selects m available clients and communicates the current global model to them. Each selected client i for $i \in \{1, 2, \ldots, m\}$ performs local model training and generates an ℓ-dimensional update vector $\mathbf{w}_i \in \mathbb{Z}_q^\ell$, where q is a large prime. The client then encodes the update vector using an additively homomorphic masking scheme before sending it to the server. The server uses secure aggregation to combine the clients' update vectors $\sum_{i=1}^{m} \mathbf{w}_i$ into a single update vector for the global model. RoFL allows the server to verify that the client-updates $\mathbf{w}_i$ are valid, i.e., fulfill a predefined constraint while preserving the guarantees of secure aggregation and remaining compatible with the conventional deployment scenario. Specifically, it allows the L_p norm of client-updates $\mathbf{w}_i$ to be constrained, i.e., $\|\mathbf{w}_i\|_p < B$ for some $B \ll q$, and we provide concrete instantiations for both L_∞ and L_2 norm bounds. Upon receiving the client input, the server verifies that the norm bound is satisfied. If the verification fails, the server discards the client input and flags the client as corrupt.

Threat Model. We make a distinction between two types of servers: malicious and semi-honest. A malicious server has the ability to deviate from the protocol arbitrarily, while a semi-honest server follows the protocol but analyzes received messages in an attempt to gain information about clients' private inputs. Our system inherits the confidentiality guarantees of secure aggregation [8, 15], which provides input privacy even in the presence of a malicious server colluding with at most a fraction γ of malicious clients. Specifically, our system ensures that an adversary cannot learn anything beyond what they can derive from the aggregation output and the proven constraint of the inputs. RoFL additionally ensures correctness against malicious clients for a semi-honest server. We assume a PKI, which allows the server to facilitate end-to-end encrypted and authenticated communication between clients without the need for direct communication between the clients.

3.3. Secure Aggregation with Commitments

We begin by providing a brief description of the secure aggregation protocol proposed by Bonawitz et al. [15] and

later extended by Bell et al. [8]. After this overview, we present our extension of the protocol, which supports private input constraints. We provide a comparison to existing work in Table 1 and an overview of the protocol in Algorithm 1 in Appendix D.

Secure Aggregation. The secure aggregation protocol is based on a homomorphic masking scheme. This involves each client blinding their input vector $\mathbf{w}_i$ with a key vector $\mathbf{r}_i$, which is carefully generated to ensure cancellation during aggregation. The distribution of masking keys $\mathbf{r}_i$ between clients is achieved by the `ShareKeys` sub-protocol. In order to support dropouts from clients during the aggregation, `Unmask` is used to reconstruct mask contributions from clients that dropped out. While RoFL extends the protocol by adapting the homomorphic masking scheme, it does not require any modifications to `ShareKeys` and `Unmask`. For a detailed description of these protocols, we, therefore, refer the reader to [8].

We abstract the method of how clients hide their vectors given a masking key as an encoding scheme in order to improve the presentation of our later extensions. This encoding scheme (`Enc`, `Dec`) should be homomorphic in both the messages $\mathbf{w}_i$ and the keys $\mathbf{r}_i$ created[4] by `ShareKeys`. Specifically, $\sum_i \texttt{Enc}(\mathbf{w}_i, \mathbf{r}_i) = \texttt{Enc}(\sum_i \mathbf{w}_i, \sum_i \mathbf{r}_i)$. The encoding scheme allows the server to aggregate messages encoded with clients' individual keys $\mathbf{r}_i$ and decrypt the aggregate $\texttt{Dec}(\sum_i \texttt{Enc}(\mathbf{w}_i, \mathbf{r}_i), \mathbf{r})$ if and only if it has access to the aggregate key $\mathbf{r} = \sum_i \mathbf{r}_i$ which can be computed securely using `Unmask`. The encoding and decoding functions of the original secure aggregation protocol are defined as:

$$\begin{aligned} \texttt{Enc}(\mathbf{w}_i, \mathbf{r}_i) &\coloneqq \mathbf{w}_i + \mathbf{r}_i \mod q \\ \texttt{Dec}(\mathbf{y}, \mathbf{r}) &\coloneqq \mathbf{y} - \mathbf{r} \mod q \end{aligned} \tag{1}$$

for $\mathbf{w}_i, \mathbf{y}, \mathbf{r}_i, \mathbf{r} \in \mathbb{Z}_q^\ell$.

Correctness with malicious clients. Prior to enforcing private constraints, it is essential that the encoding scheme allows the server to verify the correctness of the protocol, even in the presence of actively adversarial clients. The initial secure aggregation protocol only provides input privacy and does not guarantee correctness, which the authors acknowledge "is much harder to achieve" [15]. For instance, malicious clients can introduce inconsistent masking values $\mathbf{r}'_i$ during `ShareKeys` and `Unmask` leading to arbitrary outputs at the server. To establish any meaningful robustness via private constraints, it is crucial to first ensure the protocol's correctness in this scenario.

One possible approach to achieve this would be to require clients to prove that they executed $\texttt{Enc}(\mathbf{w}_i, \mathbf{r_i})$, including the generation of the $\mathbf{r}_i$, honestly as specified by the protocol. However, as the formation of $\mathbf{r}_i$ entails many evaluations of a PRG, proving this is prohibitively expensive. One of our key insights is that it is sufficient for correctness to ensure that the sum of the keys used in the encodings $\sum_i \mathbf{r}_i$ is equal to the aggregate key output by `Unmask`.

Instead, we propose an optimization that allows the server to verify this condition efficiently by exploiting the homomorphic nature of the encoding scheme. We show this optimization for a generic construction before considering how to instantiate it efficiently.

We define a new generic encoding scheme from two homomorphic masking schemes $\texttt{E}_\texttt{w}$ and $\texttt{E}_\texttt{r}$:

$$\begin{aligned} \texttt{Enc}(\mathbf{w}_i, \mathbf{r}_i) &= (\mathbf{t}_i^{(1)}, \mathbf{t}_i^{(2)}) = (\texttt{E}_\texttt{w}(\mathbf{w}_i, \mathbf{r}_i), \texttt{E}_\texttt{r}(0, \mathbf{r}_i)) \\ \texttt{Dec}((\mathbf{t}_i^{(1)}, \mathbf{t}_i^{(2)}), \mathbf{r}) &= \texttt{D}_\texttt{w}(\mathbf{t}_i^{(1)}, \mathbf{r}) \end{aligned} \tag{2}$$

Here, $\texttt{E}_\texttt{w}$ and $\texttt{E}_\texttt{r}$ must be homomorphic in both message and key, just as the original encoding scheme is. We note that, in our setting, $\texttt{E}_\texttt{r}$ must be secure even for a known message, which the original scheme does not fulfill. Depending on the instantiation, we might also require a proof that the randomness used in $\mathbf{t}_i^{(1)}$ and $\mathbf{t}_i^{(2)}$ is the same. The server can use the second element to verify if the output $\mathbf{r}$ from `Unmask` matches the sum of the keys in the encoding by checking that $\texttt{E}_\texttt{r}(0, \mathbf{r})$ is equal to $\sum_{i=1}^m \mathbf{t}_i^{(2)}$, which implies $\mathbf{r} = \sum_{i=1}^m \mathbf{r}_i$. Otherwise, the server aborts that particular round and proceeds with another subset of clients. Hence, this provides correctness for actively malicious clients by detecting malformed aggregations.

Design Space. Selecting the optimal encoding scheme boils down to finding an efficient proof system that operates on this encoding, because proof creation and verification are typically much more expensive than encoding and decoding. Clients need to prove that their updates are within the norm bound by providing range proofs for each parameter. This can be achieved via a range of state-of-the-art proof systems. For example, state-of-the-art SNARKs (e.g., Groth16 [49]) provide constant proof sizes and sub-linear verification times combined with competitive concrete prover generation times. However, in our setting, prover time is more important than verification time, as the provers (clients) are likely to be significantly more computationally constrained than the verifier (server). For RoFL, we employ Bulletproofs [20], which provide logarithmic proof sizes and linear verification times for range proofs without requiring a trusted setup. While SNARKs tend to outperform them for general proofs, Bulletproofs are highly efficient for the range proofs we require. Additionally, the proof size of Bulletproofs (while asymptotically suboptimal) is concretely very competitive: a few kilobytes suffice for a million range proofs due to efficient range proof batching (c.f. §4).

Bulletproofs are generally instantiated over Pedersen commitments [69], for which they natively support range proofs without requiring expensive emulation. While alternative homomorphic commitment schemes might outperform Pedersen commitments in terms of generation times and size, Pedersen-commitment–based Bulletproofs are an especially effective combination because of the length-reducing property of Pedersen commitments: a commitment to a vector of group elements is the same size as a commitment to a single group element. This property is

4. Technically, in [15], `ShareKeys` is used to derive seeds which are then expanded into the keys via a PRG.

vital for the sub-linear communication complexity of Bulletproofs, as highlighted by recent theoretical work [5, 17] that explores adapting ideas from Bulletproofs towards the design of an argument of knowledge that directly operates on other homomorphic commitment schemes such as lattice commitments. These works show that such an approach would likely not give the same advantages, and result in proofs that increase noticeably in size when aggregating multiple proofs.

Commitment-based Encoding. Pedersen commitments alone, however, are not sufficient to instantiate our generic encoding scheme. We instead identify ElGamal commitments [36] as the most suitable basis for our design because they can be considered as an extension of Pedersen commitments. ElGamal commitments are both computationally hiding and information-theoretically binding under a standard discrete log hardness assumption. Furthermore, ElGamal commitments are additively homomorphic in both the messages and keys, thus supporting the required homomorphisms.

We lift the masking approach from secure aggregation to the commitment-based setting by instantiating our generic encoding scheme with an ElGamal commitment $Com_{EG}(w_i, r_i)$ for each element in $\mathbf{w}_i$. This results in a vector of commitments $Com_{EG}(\mathbf{w}_i, \mathbf{r}_i)$, i.e., encoding and decoding are defined as

$$(\mathtt{E}_{\mathtt{w}}(\mathbf{w}_i, \mathbf{r}_i), \mathtt{E}_{\mathtt{r}}(0, \mathbf{r}_i)) = Com_{EG}(\mathbf{w}_i, \mathbf{r}_i) = (g^{\mathbf{w}_i} h^{\mathbf{r}_i}, g^{\mathbf{r}_i})$$
$$\mathtt{D}_{\mathtt{w}}(\mathbf{y}, \mathbf{r}) = dlog_g \left(\mathbf{y} \cdot h^{-\mathbf{r}}\right)$$

where $\mathbf{w}_i, \mathbf{w}, \mathbf{r}_i, \mathbf{r} \in \mathbb{Z}_q^\ell$ and g, h are generators in a group $\mathbb{G}$ of prime order q. The first component $g^{w_i} h^{r_i}$ is a valid Pedersen commitment to w_i on its own, which can be used directly by Bulletproofs to proof statements involving w_i. The decoding function $\mathtt{D}_{\mathtt{w}}$ computes the discrete logarithm with respect to g, which results in $\mathbf{w}$ if $\mathbf{y}$ is a correct encoding of $g^{\mathbf{w}} h^{\mathbf{r}}$. Note that we assume that the discrete log problem is hard in $\mathbb{G}$. While this means that calculating the discrete logarithm of $y = g^x$ is hard for generic $x \in \mathbb{Z}_q$, the aggregation result space in our domain is small (e.g., a 32-bit integer) and hence the discrete logarithm can nevertheless be computed efficiently [71, 79, 84]. In the presence of actively malicious clients, proofs of well-formedness are required, which we discuss in §3.4.

Instead of relying on ElGamal commitments, one could alternatively augment $g^{\mathbf{w}_i} h^{\mathbf{r}_i}$ with homomorphic message authentication codes (MACs), such as those used in generic MPC protocols like SPDZ [33]. Homomorphic MACs incur smaller bandwidth costs because the field size of the MAC can be smaller than the original group size (while providing the same level of security). However, this approach has several drawbacks when applied to the FL setting: Neither the server nor the clients must learn the MAC key, requiring a preprocessing step to generate the masking values $\mathbf{r}_i$ for each client. Moreover, checking the aggregate MAC requires an extra round of interaction after `Unmask`, which would have to be made resilient to client dropouts.

	Plaintext	Bonawitz et al. [15]	Bell et al. [8]	RoFL (§3)
Client communication	ℓ	$\ell + m$	$\ell + \log(m)$	$\ell + \log(m)$
Client computation	ℓ	ℓm	$\ell \log(m)$	$\ell \log(m)$
Input privacy	○	●	●	●
Aggregation correctness	○	○	○	●
Input validation	●	○	○	●

TABLE 1: Overview of computation and communication asymptotics and security properties for different (secure) aggregation algorithms. Big-O notation is omitted for brevity.

3.4. ZKP of Norm Bounds

In order to enforce norm bounds, RoFL requires each client to provide a NIZK proof that their update vector $\mathbf{c}_i = \mathtt{Enc}(\mathbf{w}_i, \mathbf{r}_i)$ is well formed and has a norm bounded by B, where B is set by the server. Let $\mathbf{w}_i$ be the parameter vector and $\mathbf{r}_i$ the vector of canceling nonces of client i. The clients need to provide a proof of the following relation to the server:

$$\mathcal{R} := \left\{ \left((\mathbf{w}_i, \mathbf{r}_i), \mathbf{c}_i \right) : \mathbf{c}_i = (g^{\mathbf{w}_i} h^{\mathbf{r}_i}, g^{\mathbf{r}_i}) \wedge \|\mathbf{w}_i\|_p < B \right\}$$

In the following, we first address the well-formedness proofs for the commitments before considering how to construct efficient proofs for the norm bound itself.

Well-Formedness Proofs. Clients need to proof that $\mathbf{c}_i$ is well-formed, i.e., that they used the same r_j in $g^{w_i} h^{r_j}$ and g^{r_j} for each $w_j \in \mathbf{w}_i$ and $r_j \in \mathbf{r}_i$. They do this by generating a proof-of-knowledge of an opening for each ElGamal commitment, which can be instantiated in a straightforward manner using a standard Sigma protocol [22]. The server can then check that $\prod_{i=1}^{m} g^{\mathbf{r}_i}$ is equal to $g^{\mathbf{r}}$ to ensure the randomness in the aggregate encoding is equal to $\mathbf{r}$. Directly sending the individual well-formedness proofs would introduce a bandwidth overhead of 2ℓ group elements and 2ℓ field elements per ElGamal commitment, i.e., double the size of the commitment (§4). We can reduce this overhead by combining the individual proofs for each commitment into a single proof of constant size because each proof is for the same relation, i.e., $\mathbf{c}_i = (g^{\mathbf{w}_i} h^{\mathbf{r}_i}, g^{\mathbf{r}_i})$ for the same g, h. This results in a proof of constant size, independent of the number of parameters. For a security proof of this compression, we refer to Theorem 4 in Appendix A of [48].

Norm Bounds. We now show how to realize *(i)* L_∞ and *(ii)* L_2 constraints, the two variants of norm constraints that RoFL supports. We start with the L_∞ norm bound where we can consider each parameter independently. We then show how to bound the L_2 norm, which depends on all parameters in the vector and which requires clients to additionally commit to, and prove range bounds over, the squares of each parameter. Although this introduces significant additional costs in communication and computation, L_2 bounds are frequently used in the ML robustness literature because the lack of strict bounds on each parameter allows for more uneven parameter weight distributions in the updates.

(i) L_∞-norm: To bound the L_∞ norm by B, it is sufficient for the prover to show to the verifier that each

parameter $w_j \in \mathbf{w}_i$ is within a bounded range $w_j \in [0, B)$. Bulletproofs can aggregate all ℓ required range proofs, one for each parameter update, into a single proof consisting of only five field elements and $2(\log_2(b) + \log_2(\ell)) + 4$ group elements where b is the bit length of the range. In FL, this reduces the bandwidth cost from linear in the number of parameters to logarithmic. Further, instead of mapping 32-bit floating-point parameters directly to $\mathbb{Z}_q$ with standard fixed-point encoding, RoFL compresses parameters into b-bit integers (e.g., $b = 8$ or $b = 16$) with probabilistic quantization [55]. Our evaluation in §4 shows that this quantization does not lead to a significant loss of accuracy in the overall model.

(ii) L_2-norm: Bounding each parameter by B using the L_∞ bound implies a trivial bound of the L_2 norm of $\ell \cdot B^2$. However, since an L_2 norm bound is supposed to allow more flexibility for the individual parameters, this is not a useful way to instantiate L_2 bounds. On the other hand, it is not sufficient to merely prove $\sum_{j=1}^{\ell} w_j^2 < B_{L_2}^2$. Because we are working with integers modulo q, a malicious client could cause the sum of squares to wrap around the modulus to a small value by setting one of the w_j sufficiently large to cause an overflow. Therefore, we also require a proof that each element is sufficiently small to prevent this attack. We note that each honest parameter w_j must satisfy $w_j^2 < B_{L_2}^2$ because otherwise, the sum would trivially violate the L_2 bound. Therefore, we can enforce a per-parameter bound of B_{L_2} without loss of generality. We construct our L_2 norm bounds as an extension of our L_∞ norm bounds, appending an additional proof to verify that the sum of squared parameters is bounded. More specifically, for each commitment c_j to parameter w_j, the client provides an additional Pedersen commitment to the square of the parameter, $c'_j := Com_{PD}(w_j^2, r'_j) = g^{w_j^2} h^{r'_j}$ and provides a proof that the sum of all c'_j's in the vector lies in the range $[0, B_{L_2}^2)$. The server checks this by computing the same sum over all c'_j using the homomorphic property and checking the range proof. In addition, the clients also generate a well-formedness proof to ensure that c'_j indeed commits to the square of the value committed to in c_j. We use standard non-interactive proof-of-knowledge of discrete logarithm [22] techniques by rewriting it to $c'_j = c_j^{w_j} h^{r'_j - w_j r_j}$. The prover can then use the proof-of-knowledge to show knowledge of an opening of both Pedersen commitments to the same message w_j in different bases: g and c_j. This results in one well-formedness proof per parameter. Unfortunately, we cannot apply the same compression technique used for the well-formedness proofs to combine the per-parameter proofs in a single proof of constant size. This is because the proof-of-knowledge for each commitment opening is for a different base that depends on c_j.

General Constraints. In addition to the L_∞ and L_2 norm bounds, RoFL can support arbitrary client-side constraints expressed as circuits by using standard general-purpose zero-knowledge proofs supported by Bulletproofs. However, the optimizations proposed in this paper to make the ZKPs feasible for FL workloads might not be directly compatible with arbitrary constraints, where other tailored optimizations may need to be considered.

3.5. Optimizations

Up until now, we have outlined the cryptographic components that make up RoFL and explained how a careful co-design of cryptography and the FL protocol allows our system to achieve competitive performance. Despite these improvements, scaling the protocol to accommodate for realistic deep learning models poses a challenge. Even as we optimize the cryptographic protocol to align with the specifics of FL, the sheer volume of proofs required due to the large number of model parameters inevitably leads to significant computation and bandwidth overheads. To address this challenge, we explore ways to decrease the number of proofs needed by considering optimizations that take advantage of the combination of our protocol with the underlying FL model.

Probabilistic Range-Checking (L_∞). The range proofs required for the L_∞ bound contribute significantly to the computation overhead (§4). In the base protocol, the client proves that each element in $\mathbf{w}_i$ is smaller than the bound B. However, it is not necessary to check all ℓ elements to detect a malicious client with high probability. RoFL employs probabilistic checking of a random subset of the update to reduce the number of range proofs required. The key insight is that the server can choose the random subset to verify after the clients have uploaded their commitments, taking advantage of the fact that the commitments are binding and cannot be altered. Even relatively small subsets provide strong guarantees because a single failed verification is enough to identify a malicious client. The probability of the server selecting at least one element with a malicious update exceeding the bound follows a hypergeometric distribution. By exploiting the properties of this distribution, we can make the probability of missing a violation vanishingly small. For instance, let us assume that a fraction $p_v \in (0, 1]$ of the ℓ parameters in an update exceed the bound, and the server checks a fraction $p_c \in (0, 1]$ of the parameters. We can model the probability of the server failing to detect a malicious update with $p_v \cdot \ell$ elements above the bound, while verifying $p_c \cdot \ell$ elements, as $Hyp(p_c \cdot \ell | \ell, \ell \cdot (1 - p_v), p_c \cdot \ell)$. The security guarantees of this optimization depend on the ratio of checked parameters to above-bound malicious parameters. Although it is not impossible for a successful attack to modify a very small number of parameters, even down to just one, such a hypothetical attack would have to be incredibly sophisticated, as the attacker is significantly more constrained than with existing attacks. As we apply this optimization to larger models, the fraction of checks required remains small, even as the total number of parameters increases. In Appendix B, we provide an empirical analysis of the security of probabilistic checking under generic and adaptive attacks.

This optimization cannot be applied to the L_2 norm variant because even a single parameter outside the acceptable range in the update vector could cause an overflow in the sum of squares. Essentially, this would allow an adversary

to submit an arbitrary update vector while evading detection with a high probability of success.

Compression Techniques (L_2). Because both computation and communication overheads are linear in the size of the update vectors (§4), we can benefit from ML [55] compression techniques that reduce the size of the updates. However, the need to be compatible with secure aggregation and norm bounding limits the number of applicable techniques. In RoFL, we consider random subspace learning [59] as an optimization, which was initially introduced to help understand the hardness of ML tasks. Random subspace learning applies an L_2-norm-preserving transformation that reduces the number of parameters required for a model update. Since this transformation preserves the L_2-norm, it aligns well with our L_2 constraint, making it a suitable technique for our purposes. Random subspace learning involves training the model in a lower-dimensional subspace $\mathbf{w}^d$, where d is smaller than the original model's dimension ℓ. To project this subspace $\mathbf{w}^d$ onto the original model space $\mathbf{w}^\ell$, an orthonormal projection matrix $\mathrm{P} \in \mathbb{R}^{\ell \times d}$ is used. The projection is achieved through the following formula: $\mathbf{w}^\ell = \mathrm{W}_0^\ell + \mathrm{P}\mathbf{w}^d$. During training, W_0^ℓ and P are considered as constants, and the optimization is performed on $\mathbf{w}^d$. This means that only $\mathbf{w}^d$ needs to be exchanged between the client and server in each round, resulting in a significant reduction in the number of parameters from ℓ to d. The parameter d, known as the *intrinsic dimension* [59], determines the compression ratio, and is set by the server depending on the training task.

Optimistic Continuation. The server's computation time is primarily consumed by proof verification (§4). However, proof verification does not have to block the system from running the next training round because the server already has access to the aggregation result, i.e., the global model for the next round. The clients' training usually takes significant time, depending on the complexity of the model and the size of the client dataset. In RoFL, we leverage this insight to execute client training and server proof verification in parallel. After receiving all client update vectors, the server optimistically combines the vectors, attempts to decode the sum with the aggregate key from `Unmask`, and proceeds to the next round before verifying the proofs. The server then verifies the proofs of the preceding round while the model training of the next round is already in progress. If the verification of the previous round succeeds, the server takes part in `Unmask` to reconstruct the aggregate key for the current round. Should there be any inconsistencies, e.g., a bound violation, the server aborts the new round and resets the model. However, because inconsistencies should be infrequent in practice, this optimistic approach can reduce overall wall time significantly. Note that the server waits before initiating the second phase of secure aggregation because otherwise, a malicious client could compromise the privacy of other clients' training data. In this scenario, clients could train on a model of which the integrity is not yet verified by the server, allowing a malicious client to replace the global model in aggregation with a malicious one. This setting could be exploited by applying a range of recent privacy attacks [13, 42, 68] that are able to extract training data from maliciously altered models. Delaying `Unmask` means that the server cannot decrypt the global update and thus does not gain any additional information compared to the protocol without optimistic continuation.

	MNIST	CIFAR-10 S	CIFAR-10 L	Shakespeare
Network	CNN	LeNet5 [58]	ResNet-20 [51]	LSTM [52]
Weights	19k	62k	273k	818k
Dataset	F-MNIST [21]	CIFAR-10 [56]	CIFAR-10 [56]	Shakespeare [21]
Task	image class.	image class.	image class.	text gen.

TABLE 2: Training tasks used in our evaluation.

4. Evaluation

In this section, we quantify the performance overhead of RoFL and show that it can be used to train practical ML models. It is worth noting that the cryptography used in RoFL only serves to enforce the norm bound privately, and does not impact on the efficacy of the defense itself. Therefore, our focus is to evaluate the overhead of RoFL.

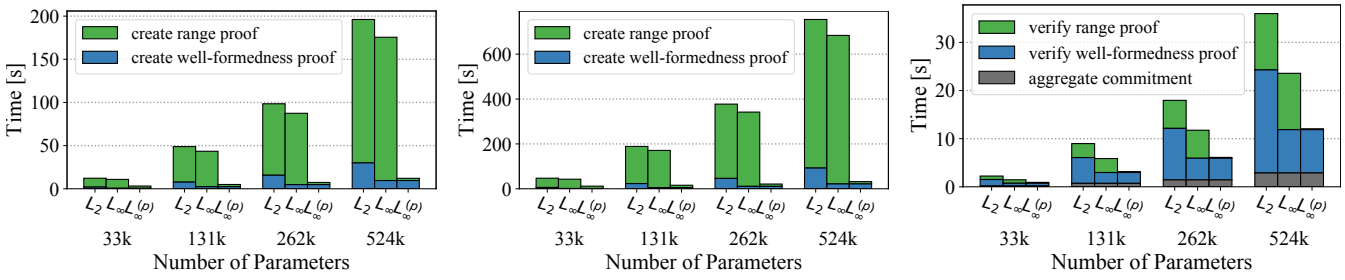
Implementation. We developed an end-to-end prototype of RoFL that we make available online. We implement the framework in Rust and interface it with Python to train neural networks with TensorFlow [2]. For client-server communication, we rely on the Tonic RPC framework [1] with protobuf. We use the elliptic curve Curve25519 (i.e., 126-bit security) implementation from the *dalek curve25519* library [32] for cryptographic operations. The library supports *avx2* and *avx512* hardware instructions for increased performance on supported platforms. The range proofs are implemented with the Bulletproof library [31], which builds on the same elliptic curve library.

Microbenchmarks. We use a single AWS EC2 instance (c5d.4xlarge, 16 vCPU, 32GiB, Ubuntu18.04) with support for *avx512* instructions. When evaluating the cryptographic overhead for the client, we limit the cryptographic library to using four parallel threads. To evaluate the performance of the microbenchmarks on constrained client devices, we also perform the experiments on a less powerful AWS EC2 instance (t2.medium, 2 vCPU, 2GiB, Ubuntu18.04). We exclude the setup cost for establishing the masking keys with Diffie-Hellman key exchanges between the clients in the microbenchmarks. We do consider this overhead in the end-to-end benchmarks.

End-to-End benchmarks. We evaluate end-to-end system performance for four tasks on the image- and text-based datasets. Table 2 provides an overview of the tasks and models we use in our evaluation. To simulate a practical deployment, we use five AWS EC2 instances (c5d.9xlarge, 36 vCPU, 72 GiB, Ubuntu18.04) with support for *avx512* instructions and with a latency of 0.5 ms between them. The server uses one instance, and 48 clients are evenly distributed over the other four instances.

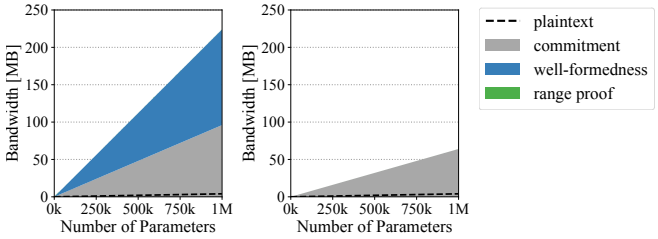
4.1. Input Validation with ZKP

We begin our evaluation by investigating the overhead of integrating ZKPs for the L_∞- or L_2-norm bound checks into secure aggregation on clients and the server. In the following, we first analyze the computation and bandwidth costs of the baseline protocol and then show the benefits



(a) large client (b) small client (c) server: per client

Figure 12: Microbenchmark results. Client computation cost for creating the L_2/L_∞-norm bound proof (a, b). Server computation for ZKP verification, commitment aggregation (c).



(a) L_2 BW (b) L_∞ / (p) BW

Figure 13: Bandwidth cost per client to upload the update to the server. Range proofs' overhead is negligible compared to that of commitments and well-formedness proofs.

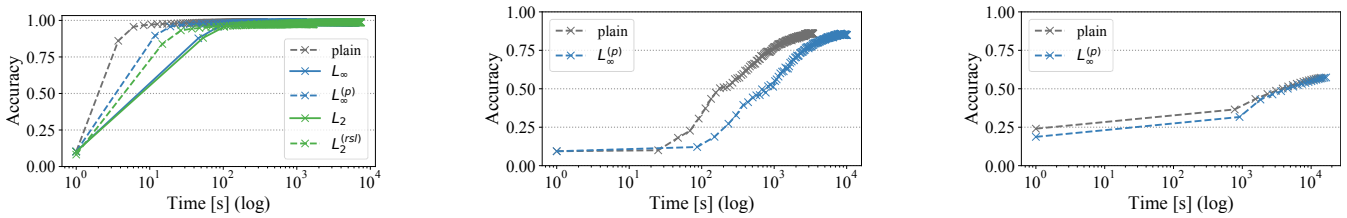
of using our probabilistic checking optimization. We defer discussion of the subspace learning optimization for L_2 to the end-to-end experiments.

Computation. The client-side computational costs, beyond the FL protocol itself, only include computing commitments to each parameter and creating the corresponding zero-knowledge norm proofs. This additional cost is linear in the number of parameters, as seen in Figs. 12a and 12b. On a strong client (Fig. 12a), the creation of L_2 and L_∞ proofs for 262k parameters takes 98 and 87 seconds, respectively, whereas on a less powerful machine (Fig. 12b), this cost increases by 3x to 377 and 342 seconds. These costs are predominately determined by the range-proof creation, to which the additional well-formedness proofs to ensure correct encoding do not contribute significantly. Although these costs are significant, they are within an order of magnitude of the model training cost, so clients that can perform the underlying FL computation efficiently should be able to handle the overhead too.

When using L_2-norm proofs, clients need to compute additional proof-of-squares, which adds a minor overhead compared to L_∞-norm proofs. Meanwhile, the server side verifies the proofs of each client, aggregates them into the global vector, and reconstructs the aggregated vector once all updates are included. Similar to the client-side costs, the server-side verification and aggregation costs increase linearly with the number of parameters, as shown in Fig. 12c. With an update size of 262k, the server requires 12–18s per client to verify the proofs and aggregate the commitments, where the verification of the range proofs dominates the processing time. Once the clients' commitments are aggregated, the server has to solve the discrete-log for each parameter. The discrete-log solving time increases linearly with the number of parameters (i.e., 467ms for 262k and 627ms for 524k params), but the overhead is marginal compared to proof verification costs. The tasks at the server are highly parallelizable, so we can reduce the overall computation time by horizontal scaling by adding more processors.

Communication. In Figs. 13a and 13b, we show that the bandwidth requirements grow linearly in the number of parameters. The bandwidth costs are dominated by size of the commitments because RoFL uses an ElGamal commitment consisting of two 256-bit group elements per 32-bit parameter to ensure the aggregate value can be computed using the discrete log. The bandwidth costs are higher for the L_2-norm (Fig. 13a) than the L_∞-norm (Fig. 13b), due to the additional commitments to the square and the corresponding proofs. The range proofs can be aggregated into a single proof, whose size increases logarithmically in the number of parameters, but for computational efficiency in our evaluation we aggregate the proofs into 16 proofs to improve parallelism. These proofs are concretely very small: approximately 24KB for a vector of 262k parameters. In addition, the compressed well-formedness proof has a constant size of only two field and two group elements. Thus, the bandwidth costs are dominated by the commitment size and, for the L_2-norm, the additional group elements required for the well-formedness proof of the square commitments. For 262k parameters, the data transmitted to the server per client is 59MB (56x compared to plaintext) for the L_2-norm setting and 17MB (16x) for the L_∞-norm setting. Clearly, this is a significant overhead that limits the size of the model that this technique can scale to. However, we note that while the relative size increase is dramatic, the absolute size of the updates is about the same as a few minutes of compressed HD video and, therefore, still well within the capabilities of most modern connections, including wireless and mobile.

Probabilistic Checking. We now show the performance improvements of probabilistic checking of L_∞ constraints. We set a maximum failure probability of 10^{-8} and assume that at least $p_v = 0.005$ of the parameters in a malicious update must exceed the bound for an attack to be successful. We provide a more thorough justification, including empirical evidence, for these choices in Appendix §B. Bandwidth is negligibly affected by probabilistic checking because the aggregated range-proofs are already logarithmically sized (Fig. 13b). However, probabilistic checking can reduce the



(a) MNIST (b) CIFAR-10 L (c) Shakespeare

Figure 14: Model convergence over time.

computation cost by 12x–17x for the clients and 2x for the server (Fig. 12). The optimization shows better improvements when using larger numbers of parameters: for instance, from 87s to 7s on the large client and 342s to 21s per client server-side for 262k parameters. We conclude that probabilistic checking dramatically reduces the overhead of applying L_∞-norm constraints while offering almost the same level of protection.

4.2. RoFL End-to-End Performance

We demonstrate the effectiveness of RoFL in a practical deployment by showing that RoFL provides a significant speedup in computation time compared to the unoptimized L_∞- and L_2-norm constraints. For the optimized version of the L_∞-norm, we apply probabilistic checking (denoted by $L_\infty^{(p)}$, and for L_2, we compress the updates with random subspace learning (rsl, denoted by $L_2^{(rsl)}$) [59]. We also show the feasibility of deploying RoFL for practical models with a large number of parameters.

RoFL L_2 vs Secure Aggregation. We set the effective number of parameters in L_2-norm with rsl, i.e., the intrinsic dimension, for `MNIST` to 5000 (3.8x parameter reduction), for `CIFAR-10 S` to 12000 (5.2x parameter reduction), and for `CIFAR-10 L` to 40000 (6.8x parameter reduction) as explored by Li et al. [59]. Table 3 compares the results of the different runs. In the `MNIST` task, our optimization reduces the training time per round from 21x to 5x compared to secure aggregation. In the `CIFAR-10 S` and `CIFAR-10 L` experiments, we find that RoFL reduces the computation per round from 60x to 10x and from 36x to 7x. Quantization slightly reduces the accuracy for `CIFAR-10 S` and `CIFAR-10 L`, from 0.61 for plaintext to 0.60 for `CIFAR-10 S`, and from 0.86 to 0.85 for `CIFAR-10 L`. In addition, we observe a small impact on accuracy in all tasks due to the subspace ML compression, which is consistent with the work by Li et al. [59]. Moreover, the compression reduces bandwidth costs from 28x to 6x for `CIFAR-10 S` and from 28x to 4x for `CIFAR-10 L` compared to the secure aggregation baseline.

RoFL L_∞ vs Secure Aggregation. We show the use of probabilistic checking to optimize the L_∞-norm constraint for various model sizes. Similar to the microbenchmarks, we set a maximum failure probability of 10^{-8} in the optimized experiments while assuming $p_v = 0.005$ The optimization reduces the computation overhead in RoFL per round from 18x to 3x for `MNIST` and from 47x to 8x for `CIFAR-10 S` (Table 3). The probabilistic checking optimization shows benefits even for smaller models, but it shows the most significant speedups when applied to larger models (e.g., `CIFAR-10 L` and `Shakespeare`). Although bandwidth use remains similar to unoptimized RoFL, probabilistic checking reduces the computation overhead for `CIFAR-10 L` by more than one order of magnitude: from 31x to 2.9x compared to secure aggregation. Meanwhile, for `Shakespeare`, it reduces the computation overhead from 2x to 1.2x. Because this model has a higher ratio of training computation to the number of parameters, the cryptographic overhead of RoFL is smaller. The larger number of parameters allows probabilistic checking to achieve an even greater relative speedup here. Model convergence is similar for the secure aggregation and, as shown in Fig. 14, accuracy reductions due to quantization are less than 1%. As a result, although RoFL introduces a noticeable overhead, especially in terms of bandwidth, computation and communication in RoFL remain efficient enough to be practical for real-world deployments.

Type	Acc.	Computation Time			Bandwidth	
		Round [s]	Total [m]	Factor	Total [MB]	Factor
MNIST (19k params, rsl 5k params, 160 rounds)						
SA	0.99	2	6	1x	0.2	1x
L_2	0.99	47	126	21x	4.5	26x
$L_2^{(rsl)}$	0.97	11	28	5x	1.2	7x
L_∞	0.99	40	108	18x	1.4	8x
$L_\infty^{(p)}$	0.99	8	21	3x	1.4	8x
CIFAR-10 S (62k params, rsl 12k params, 100 rounds)						
SA	0.61	2	3	1x	0.5	1x
L_2	0.60	110	184	60x	14.2	28x
$L_2^{(rsl)}$	0.59	18	30	10x	2.8	6x
L_∞	0.60	86	144	47x	4.3	8x
$L_\infty^{(p)}$	0.61	15	24	8x	4.3	8x
CIFAR-10 L (273k params, rsl 40k params, 160 rounds)						
SA	0.86	22	58	1x	2.2	1x
L_2 *	0.85	800	2133	36x	62.4	28x
$L_2^{(rsl)}$	0.82	149	398	7x	9.2	4x
L_∞ *	0.85	688	1835	31x	18.7	8x
$L_\infty^{(p)}$	0.85	64	169	2.9x	18.6	8x
SHAKESPEARE (818k params, 20 rounds)						
SA	0.57	773	245	1x	6.6	1x
L_2 *	0.57	2526	842	3x	186.6	28x
L_∞ *	0.57	2397	799	3x	55.7	8x
$L_\infty^{(p)}$	0.57	918	291	1.2x	55.7	8x

TABLE 3: E2E model training with RoFL (* extrapolated). Bandwidth is reported per client per round.

Optimistic Continuation. Fig. 15 shows the effect of optimistic continuation. The server utilizes the idle time by verifying proofs lazily after starting the next round. The optimization results in a speedup of 40% for `CIFAR-10 L` and 48% for `CIFAR-10 S` by parallelizing proof verification with model training and proof creation. For `Shakespeare`, `CIFAR-10 L`, and `MNIST`, proof verifi-

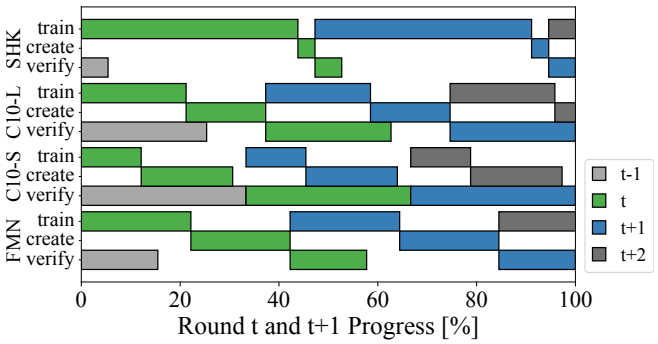


Figure 15: Optimistic continuation allows the parallelization of client-side operations with server-side proof verification.

cation requires no additional wall-clock time because model training and proof creation in $t+1$ take longer than the proof verification of t round. Only for `CIFAR-10 S`, proof verification remains on the critical path.

5. Related Work

In the interest of space, we review related work solely on cryptographic systems similar to RoFL; we have discussed the wider field of related work on FL robustness in §2.

Secure Data Collection. Secure aggregation systems [3, 23, 26, 29, 57, 61, 62, 65, 72, 75, 78, 84] enable users to compute sums over their private data without disclosing the inputs. A variety of secure aggregation protocols have emerged to meet the needs of a wide range of applications that introduce new security, scalability, and performance requirements. Bonawitz et al. [15] and Bell et al. [8] introduce a secure aggregation protocol designed for FL based on canceling random masks; their protocol builds on [3, 65]. However, none of these systems address the correctness or integrity of the computation.

Prior work has explored systems that aggregate additively homomorphic ciphertexts [3, 37, 44, 84] and commitments [47, 57, 76]. The encoding used in RoFL is most similar to the one proposed in the protocol by Shi et al. [84]. However, our protocol is tailored to work with FL, and, unlike prior work, supports correctness against malicious clients and does not assume a trusted setup. Kursawe et al. [57] extend a variation of the protocol by Shi et al. [84] with ZKP to provide correct execution guarantees but do not constrain client inputs, and their construction does not scale well to FL workloads. Canny et al. [23] propose a private peer-to-peer machine learning platform that allows clients to verify the L_2-norm of inputs. However, it exhibits high overheads that make it unsuitable for the FL setting. A different body of work in this space [4, 16, 29, 30, 35] considers a non-colluding trust model in which the clients split the input values into shares and submit them to non-colluding servers. This enables efficient input checks, but requires a significant change in the trust model [34].

FL with Differential Privacy. RoFL opts for a cryptographic solution to hide client updates from the server. An alternative approach to client update privacy is to employ differentially-private mechanisms that perturb the local client updates [46, 64, 70, 88]. However, a pure perturbation-based approach for privacy in the FL setting that guarantees local differential privacy (i.e., hiding updates) requires clients to add a substantial amount of noise [46, 70, 88], significantly reducing utility, making it impractical in many situations. RoFL composes well with differential-privacy techniques for output privacy by enforcing norm bounds on clients' updates but one must navigate the trade-off between utility, privacy, fairness, and robustness (§2.5). Still, we believe that perturbation-based differential privacy can be an orthogonal feature that RoFL can be combined with to increase robustness [7, 66, 86, 90] and privacy further.

6. Conclusion

Federated Learning is no longer widely believed to provide sufficient privacy benefits, as the update vectors still leak considerable information [13, 45, 95]. As a result, secure federated learning systems are rapidly emerging as the preferred choice for preserving privacy in the federated learning setting. The scalability challenges intrinsic to FL have given rise to new secure aggregation protocols tailored to accommodate the scalability challenges of real-world FL settings. However, strengthening the robustness of these protocols while maintaining scalability continues to pose a significant challenge. Straightforward approaches tend to become infeasible well before attaining the desired scale of FL deployment. This paper presented RoFL, a system that allows a server to enforce input constraints on clients' (private) update vectors for the FL settings to strengthen robustness. RoFL achieves practicality even at a large scale. However, it incurs considerable overhead in terms of computational resources and, notably, bandwidth. While L_∞ norm bounds can be enforced efficiently for unmodified ML systems in RoFL, achieving acceptable results for L_2 norm bounds at the scale required adapting the underlying learning process. Removing the need to adapt existing systems would allow easier adoption of robust FL and therefore represent an important direction for future work. In terms of overhead, the high dimensionality of FL data means that applying existing ZKP solutions frequently means that achieving faster proof generation comes at prohibitively increased bandwidth requirements. One possible avenue to address this might be through assuming a split-trust model with non-colluding servers, though it remains unclear how feasible this setting is in practice [34]. Alternatively, novel solutions that can efficiently aggregate commitments and range proofs to reduce bandwidth while maintaining competitive prover times present an interesting direction for future work.

Acknowledgments

We would like to thank Christian Knabenhans for his help in evaluating RoFL. We also thank Marko Mihajlovic, Matthias Lei, Emanuel Opel, Kenny Paterson, and the PPS Lab team for their insightful input and feedback. We would also like to acknowledge our sponsors for their generous support, including Meta, Google, SNSF through an Ambizione Grant No. 186050, and the Semiconductor Research Corporation.

References

[1] "Tonic RPC framework," Online: https://github.com/hyperium/tonic, May 2021.

[2] M. Abadi *et al.*, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org. [Online]. Available: http://tensorflow.org/

[3] G. Ács and C. Castelluccia, "I have a DREAM! (DiffeRentially private smart metering)," in *Information Hiding*. Springer Berlin Heidelberg, 2011, vol. 6958, pp. 118–132.

[4] S. Addanki, K. Garbe, E. Jaffe, R. Ostrovsky, and A. Polychroniadou, "Prio+: Privacy Preserving Aggregate Statistics via Boolean Shares," *Cryptology ePrint Archive cryptoeprint:2021/576*, 2021.

[5] M. R. Albrecht and R. W. F. Lai, "Subtractive sets over cyclotomic rings: Limits of schnorr-like arguments over lattices," *Cryptology ePrint Archive*, 2021. [Online]. Available: https://eprint.iacr.org/2021/202.pdf

[6] E. Bagdasaryan, O. Poursaeed, and V. Shmatikov, "Differential privacy has disparate impact on model accuracy," in *NeurIPS*, 2019.

[7] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How To Backdoor Federated Learning," in *AISTATS*, 2020, pp. 2938–2948.

[8] J. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, "Secure Single-Server Aggregation with (Poly)Logarithmic Overhead," in *ACM CCS*, 2020.

[9] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, "Analyzing federated learning through an adversarial lens," in *ICML*, 2019, pp. 634–643.

[10] B. Biggio, B. Nelson, and P. Laskov, "Poisoning Attacks against Support Vector Machines," in *ICML*, 2012, p. 1467–1474.

[11] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent," in *NeurIPS*, 2017, pp. 119–129.

[12] M. Blum, P. Feldman, and S. Micali, "Non-interactive zero-knowledge and its applications," in *ACM STOC*. ACM, 1 Jan. 1988, pp. 103–112. [Online]. Available: http://dx.doi.org/10.1145/62212.62222

[13] F. Boenisch, A. Dziedzic, R. Schuster, A. S. Shamsabadi, I. Shumailov, and N. Papernot, "When the curious abandon honesty: Federated learning is not private," *arXiv preprint arXiv:2112.02918*, 2021.

[14] J. Böhler and F. Kerschbaum, "Secure Multi-party Computation of Differentially Private Median," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 2147–2164.

[15] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for Privacy-Preserving machine learning," in *ACM CCS*, 2017, pp. 1175–1191.

[16] D. Boneh, E. Boyle, H. Corrigan-Gibbs, N. Gilboa, and Y. Ishai, "Lightweight Techniques for Private Heavy Hitters," in *IEEE Symposium on Security and Privacy (SP)*, 2021.

[17] J. Bootle, V. Lyubashevsky, N. K. Nguyen, and G. Seiler, "A non-PCP approach to succinct Quantum-Safe Zero-Knowledge," *Cryptology ePrint Archive*, 2020. [Online]. Available: https://eprint.iacr.org/2020/737.pdf

[18] D. R. Brendan McMahan, "Federated Learning: Collaborative Machine Learning without Centralized Training Data," Online: https://ai.googleblog.com/2017/04/federated-learning-collaborative.html, 2017.

[19] T. S. Brisimi, R. Chen, T. Mela, A. Olshevsky, I. C. Paschalidis, and W. Shi, "Federated learning of predictive models from federated electronic health records," *Int. J. Med. Inform.*, vol. 112, pp. 59–67, Apr. 2018.

[20] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *IEEE Symposium on Security and Privacy (SP)*, May 2018, pp. 315–334. [Online]. Available: http://dx.doi.org/10.1109/SP.2018.00020

[21] S. Caldas, P. Wu, T. Li, J. Konečnỳ, H. B. McMahan, V. Smith, and A. Talwalkar, "Leaf: A benchmark for federated settings," *arXiv preprint arXiv:1812.01097*, 2018.

[22] J. Camenisch and M. Stadler, "Proof systems for general statements about discrete logarithms," *Technical report/Dept. of Computer Science, ETH Zürich*, vol. 260, 1997.

[23] J. Canny, "Collaborative Filtering with Privacy," in *IEEE Symposium on Security and Privacy (SP)*, 2002, pp. 45–57.

[24] N. Carlini, C. Liu, Ú. Erlingsson, J. Kos, and D. Song, "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks," in *USENIX Security*, 2019.

[25] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in *USENIX Security*, 2021.

[26] C. Castelluccia, A. C.-F. Chan, E. Mykletun, and G. Tsudik, "Efficient and provably secure aggregation of encrypted data in wireless sensor networks," *ACM Trans. Sen. Netw.*, vol. 5, no. 3, pp. 20:1–20:36, Jun. 2009. [Online]. Available: http://doi.acm.org/10.1145/

1525856.1525858

[27] L. Chen, H. Wang, Z. B. Charles, and D. S. Papailiopoulos, "DRACO: byzantine-resilient distributed training via redundant gradients," in *ICML*, 2018, pp. 902–911. [Online]. Available: http://proceedings.mlr.press/v80/chen18l.html

[28] X. Chen, C. Liu, B. Li, K. Lu, and D. Song, "Targeted backdoor attacks on deep learning systems using data poisoning," *arXiv preprint arXiv:1712.05526*, 2017.

[29] H. Corrigan-Gibbs and D. Boneh, "Prio: private, robust, and scalable computation of aggregate statistics," in *USENIX NSDI*, 2017, pp. 259–282.

[30] H. N. Corrigan-Gibbs, "Protecting Privacy by Splitting Trust," Ph.D. dissertation, Stanford University, 2019.

[31] D. Cryptography, "Rust Bulletproofs Library," Online: https://github.com/dalek-cryptography/bulletproofs, August 2020.

[32] ——, "Rust Curve25519 Library," Online: https://github.com/dalek-cryptography/curve25519-dalek, August 2020.

[33] I. Damgård, V. Pastro, N. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *Advances in Cryptology – CRYPTO 2012*. Springer Berlin Heidelberg, 2012, pp. 643–662. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-32009-5_38

[34] E. Dauterman, V. Fang, N. Crooks, and R. A. Popa, "Reflections on trusting distributed trust," in *Proceedings of the 21st ACM Workshop on Hot Topics in Networks*, ser. HotNets '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 38–45. [Online]. Available: https://doi.org/10.1145/3563766.3564089

[35] Y. Duan, N. Youdao, J. F. Canny, and J. Z. Zhan, "P4P: Practical Large-Scale Privacy-Preserving distributed computation robust against malicious users," in *USENIX Security*, 2010, pp. 207–222. [Online]. Available: http://www.usenix.org/events/sec10/tech/full_papers/Duan.pdf

[36] T. ElGamal, "A public key cryptosystem and a signature scheme based on discrete logarithms," in *Advances in Cryptology*, G. R. Blakley and D. Chaum, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1985, pp. 10–18.

[37] Z. Erkin and G. Tsudik, "Private computation of spatial and temporal power consumption with smart meters," in *Applied Cryptography and Network Security*, F. Bao, P. Samarati, and J. Zhou, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 561–577.

[38] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to Byzantine-robust federated learning," in *USENIX Security*, 2020, pp. 1605–1622.

[39] V. Feldman, "Does learning require memorization? a short tale about a long tail," in *ACM STOC*, 2020, p. 954–959.

[40] V. Feldman and C. Zhang, "What neural networks memorize and why: Discovering the long tail via influence estimation," in *NeurIPS*, 2020, pp. 2881–2891.

[41] A. Fiat and A. Shamir, "How to Prove Yourself: Practical Solutions to Identification and Signature Problems," in *CRYPTO*, 1987, pp. 186–194.

[42] L. Fowl, J. Geiping, W. Czaja, M. Goldblum, and T. Goldstein, "Robbing the fed: Directly obtaining private data in federated learning with modified models," October 2021. [Online]. Available: http://arxiv.org/abs/2110.13057

[43] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "Mitigating Sybils in Federated Learning Poisoning," *arXiv preprint arXiv:1808.04866*, 2018.

[44] F. D. Garcia and B. Jacobs, "Privacy-friendly energy-metering via homomorphic encryption," in *Security and Trust Management*, J. Cuellar, J. Lopez, G. Barthe, and A. Pretschner, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 226–238.

[45] J. Geiping, H. Bauermeister, H. Droge, and M. Moeller, "Inverting Gradients - How Easy is It to Break Privacy in Federated Learning?" in *NeurIPS*, 2020.

[46] R. C. Geyer, T. Klein, and M. Nabi, "Differentially Private Federated Learning: A Client Level Perspective," *arXiv preprint arXiv:1712.07557*, 2017.

[47] J. Groth, "Non-interactive Zero-Knowledge arguments for voting," in *Applied Cryptography and Network Security*. Springer Berlin Heidelberg, 2005, pp. 467–482. [Online]. Available: http://dx.doi.org/10.1007/11496137_32

[48] ——, "Linear algebra with sub-linear Zero-Knowledge arguments," in *Advances in Cryptology - CRYPTO 2009*. Springer Berlin Heidelberg, 2009, pp. 192–208. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-03356-8_12

[49] ——, "On the size of pairing-based non-interactive arguments," in *EUROCRYPT*. Springer, 2016, pp. 305–326.

[50] T. Gu, B. Dolan-Gavitt, and S. Garg, "Badnets: Identifying vulnerabilities in the machine learning model supply chain," *arXiv preprint arXiv:1708.06733*, 2017.

[51] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *arXiv preprint arXiv:1512.03385*, 2015.

[52] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, p. 1735–1780, nov 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[53] Y. Ji, X. Zhang, S. Ji, X. Luo, and T. Wang, "Model-reuse attacks on deep learning systems," in *ACM CCS*, 2018, pp. 349–363.

[54] P. Kairouz *et al.*, "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1–2, p. 1–210, jun 2021. [Online]. Available: https://doi.org/10.1561/2200000083

[55] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtarik, A. T. Suresh, and D. Bacon, "Federated Learning: Strategies for Improving Communication Efficiency," in *NeurIPS Workshop on Private Multi-Party Machine Learning*, 2016.

[56] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[57] K. Kursawe, G. Danezis, and M. Kohlweiss, "Privacy-Friendly aggregation for the Smart-Grid," in *Privacy Enhancing Technologies*. Springer Berlin Heidelberg, 2011, pp. 175–191. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-22263-4_10

[58] Y. LeCun *et al.*, "LeNet-5, convolutional neural networks," http://yann.lecun.com/exdb/lenet/, 2015.

[59] C. Li, H. Farkhoor, R. Liu, and J. Yosinski, "Measuring the Intrinsic Dimension of Objective Landscapes," in *ICLR*, 2018.

[60] Y. Liu, S. Ma, Y. Aafer, W.-C. Lee, J. Zhai, W. Wang, and X. Zhang, "Trojaning Attack on Neural Networks," in *NDSS*, 2018.

[61] Lukas Burkhalter and Anwar Hithnawi and Alexander Viand and Hossein Shafagh and Sylvia Ratnasamy, "TimeCrypt: Encrypted Data Stream Processing at Scale with Cryptographic Access Control," in *USENIX NSDI*, 2020.

[62] Lukas Burkhalter, Nicolas Kuchler, Alexander Viand, Hossein Shafagh, Anwar Hithnawi, "Zeph: Cryptographic Enforcement of End-to-End Data Privacy," in *USENIX OSDI*, 2021.

[63] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-Efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.

[64] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *6th International Conference on Learning Representations, ICLR 2018*, 2018.

[65] L. Melis, G. Danezis, and E. De Cristofaro, "Efficient private statistics with succinct sketches," in *NDSS*, 2016.

[66] M. Naseri, J. Hayes, and E. De Cristofaro, "Local and central differential privacy for robustness and privacy in federated learning," in *NDSS*, 2022.

[67] X. Pan, M. Zhang, D. Wu, Q. Xiao, S. Ji, and Z. Yang, "Justinian's GAAvernor: Robust Distributed Learning with Gradient Aggregation Agent," in *USENIX Security*, 2020, pp. 1641–1658.

[68] D. Pasquini, D. Francati, and G. Ateniese, "Eluding secure aggregation in federated learning via model inconsistency," November 2021. [Online]. Available: http://arxiv.org/abs/2111.07380

[69] T. P. Pedersen, "Non-Interactive and Information-Theoretic secure verifiable secret sharing," in *CRYPTO*. Springer Berlin Heidelberg, 1992, pp. 129–140.

[70] V. Pihur, A. Korolova, F. Liu, S. Sankuratripati, M. Yung, D. Huang, and R. Zeng, "Differentially-Private "Draw and Discard" Machine Learning," *arXiv preprint arXiv:1807.04369*, 2018.

[71] J. M. Pollard, "Monte carlo methods for index computation (mod p)," *Mathematics of Computation*, vol. 32, no. 143, p. 918, Jul. 1978. [Online]. Available: http://dx.doi.org/10.2307/2006496

[72] R. A. Popa, A. J. Blumberg, H. Balakrishnan, and F. H. Li, "Privacy and accountability for location-based aggregate statistics," in *ACM CCS*, 2011, pp. 653–666.

[73] S. Rajput, H. Wang, Z. Charles, and D. Papailiopoulos, "Detox: A redundancy-based framework for faster and more robust gradient aggregation," in *NeurIPS*, 2019.

[74] S. Ramaswamy, R. Mathews, K. Rao, and F. Beaufays, "Federated Learning for Emoji Prediction in a Mobile Keyboard," *arXiv preprint arXiv:1906.04329*, 2019.

[75] L. Reyzin, A. Smith, and S. Yakoubov, "Turning HATE Into LOVE: Compact Homomorphic Ad Hoc Threshold Encryption for Scalable MPC," Cryptology ePrint Archive, Report 2018/997, 2018, https://ia.cr/2018/997.

[76] A. Rial and G. Danezis, "Privacy-preserving smart metering," in *Proceedings of the 10th Annual ACM Workshop on Privacy in the Electronic Society*, ser. WPES '11. New York, NY, USA: Association for Computing Machinery, 2011, p. 49–60. [Online]. Available: https://doi.org/10.1145/2046556.2046564

[77] N. Rieke *et al.*, "The future of digital health with federated learning," *NPJ Digit Med*, vol. 3, p. 119, Sep. 2020.

[78] E. Roth, D. Noble, B. H. Falk, and A. Haeberlen, "Honeycrisp: large-scale differentially private aggregation without a trusted core," in *ACM SOSP*, 2019, pp. 196–210.

[79] H. Shafagh, A. Hithnawi, L. Burkhalter, P. Fischli, and S. Duquennoy, "Secure sharing of partially homomorphic encrypted IoT data," in *ACM SenSys*, 2017, pp. 29:1–29:14.

[80] V. Shejwalkar, A. Houmansadr, P. Kairouz, and D. Ramage, "Back to the Drawing Board: A Critical Eval-

uation of Poisoning Attacks on Production Federated Learning," in *IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 1117–1134.

[81] M. J. Sheller, B. Edwards, G. A. Reina, J. Martin, S. Pati, A. Kotrotsou, M. Milchenko, W. Xu, D. Marcus, R. R. Colen, and S. Bakas, "Federated learning in medicine: facilitating multi-institutional collaborations without sharing patient data," *Sci. Rep.*, vol. 10, no. 1, p. 12598, Jul. 2020.

[82] M. J. Sheller, G. A. Reina, B. Edwards, J. Martin, and S. Bakas, "Multi-Institutional deep learning modeling without sharing patient data: A feasibility study on brain tumor segmentation," *Brainlesion*, vol. 11383, pp. 92–104, Jan. 2019.

[83] S. Shen, S. Tople, and P. Saxena, "Auror: Defending against poisoning attacks in collaborative deep learning systems," in *ACM ACSAC*, 2016, pp. 508–519.

[84] E. Shi, H. T. H. Chan, E. Rieffel, R. Chow, and D. Song, "Privacy-preserving aggregation of time-series data," in *NDSS*, 2011.

[85] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *IEEE Symposium on Security and Privacy (SP)*, 2017, pp. 3–18.

[86] A. T. Suresh, B. McMahan, P. Kairouz, and Z. Sun, "Can you really backdoor federated learning?" in *Federated learning workshop at NeurIPS*, 2019.

[87] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," in *Computer Security – ESORICS 2020*. Springer International Publishing, 2020, pp. 480–501. [Online]. Available: http://dx.doi.org/10.1007/978-3-030-58951-6_24

[88] S. Truex, L. Liu, K.-H. Chow, M. E. Gursoy, and W. Wei, "LDP-Fed: Federated learning with local differential privacy," in *Proceedings of the Third ACM International Workshop on Edge Systems, Analytics and Networking*, 2020, pp. 61–66.

[89] G. Van Horn and P. Perona, "The Devil is in the Tails: Fine-grained Classification in the Wild," *arXiv preprint arXiv:1709.01450*, 2017.

[90] H. Wang, K. Sreenivasan, S. Rajput, H. Vishwakarma, S. Agarwal, J.-y. Sohn, K. Lee, and D. Papailiopoulos, "Attack of the Tails: Yes, You Really Can Backdoor Federated Learning," in *NeurIPS*, 2020, pp. 16 070–16 084.

[91] Y. Wen, J. Geiping, L. Fowl, H. Souri, R. Chellappa, M. Goldblum, and T. Goldstein, "Thinking two moves ahead: Anticipating other users improves backdoor attacks in federated learning," 2022. [Online]. Available: https://arxiv.org/abs/2210.09305

[92] C. Xie, K. Huang, P.-Y. Chen, and B. Li, "DBA: Distributed Backdoor Attacks against Federated Learning," in *ICLR*, 2020.

[93] C. Xie, O. Koyejo, and I. Gupta, "Fall of empires: Breaking byzantine-tolerant SGD by inner product manipulation," in *UAI*, vol. 115, 2020, pp. 261–270.

[94] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates," in *ICML*, 2019.

[95] H. Yin, A. Mallya, A. Vahdat, J. M. Alvarez, J. Kautz, and P. Molchanov, "See through gradients: Image batch recovery via gradinversion," in *CVPR*, 2021.

[96] C. Zhang, S. Li, X. Junzhe, , W. Wang, F. Yan, and L. Yang, "BatchCrypt: Efficient Homomorphic Encryption for Cross-Silo Federated Learning," in *USENIX ATC*, 2020.

[97] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning requires rethinking generalization," in *ICLR*, 2017.

[98] Z. Zhang, A. Panda, L. Song, Y. Yang, M. Mahoney, P. Mittal, R. Kannan, and J. Gonzalez, "Neurotoxin: Durable backdoors in federated learning," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 26 429–26 446. [Online]. Available: https://proceedings.mlr.press/v162/zhang22w.html

[99] X. Zhu, D. Anguelov, and D. Ramanan, "Capturing long-tail distributions of object subcategories," in *IEEE CVPR*, 2014, pp. 915–922.

Appendix A.

We present additional experiments for the robustness analysis in this appendix.

Untargeted attacks. In addition to targeted attacks, we consider the performance of untargeted attacks for various norm bounds. We use the state-of-the-art Projected Gradient Ascent (PGA) attack proposed by Shejwalkar et al. [80] in our setup. The authors show the performance of this attack for different fractions of compromised clients. We take an orthogonal approach and study the impact of the choice of norm bound on the attack. We use the same setup as for targeted attacks which allows the attacker to perform an attack for 500 rounds. We define *Attack Impact* as the difference between the accuracy of the unattacked global model and the attacked model after 500 rounds. To account for per-round variance of model performance, we compute the accuracy as the average over the last 20 rounds. For tight norm bounds (**FMN**: L_2-B < 10, r-M < 15, L_∞-B < 0.1, **C10**: L_2-B < 20, r-M < 15, L_∞-B < 0.05), untargeted attack impact is negligible (i.e., less than 6%, Fig. 16). As the bound loosens, the attack impact becomes noticeable. However, when compared to targeted attacks, untargeted attacks are only effective under a much larger bound than targeted attacks. For instance, untargeted attacks start to become effective for a median-based bound r-M

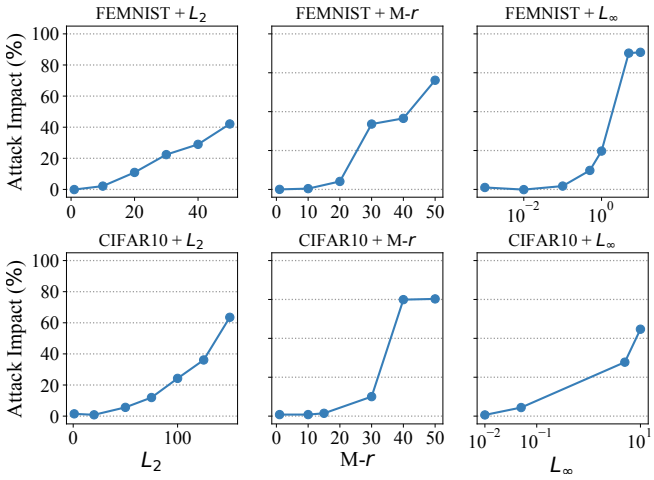
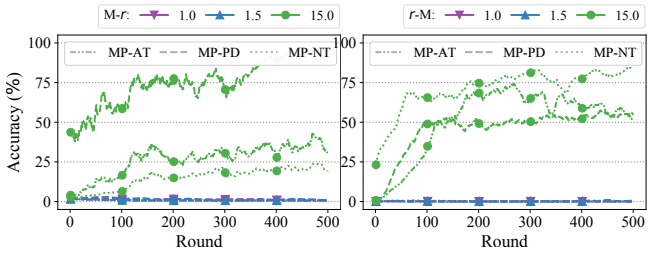


Figure 16: Attack impact of the state-of-the-art untargeted PGA attack.



(a) **FMN-P** (b) **C10-P**

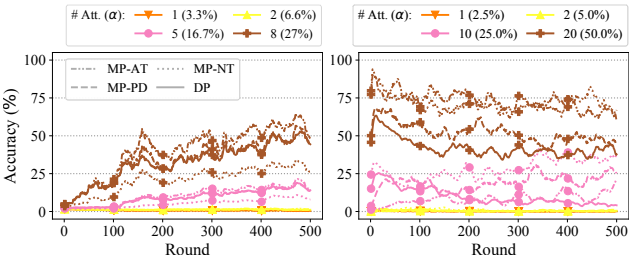
Figure 17: Comparison of model poisoning attacks under various median bounds.

$>= 30$, whereas targeted attacks already have significant success for r-M $>= 15$ (Figs. 4c and 4d). This suggests that strong untargeted attacks are easier to prevent with a norm bound than targeted attacks.

Comparison of model poisoning attacks. We evaluate the three model poisoning attack strategies (MP-AT, MP-PD, MP-NT) under adaptive norm bounds in Fig. 17 to complement the comparison under static bounds (Fig. 5 in the analysis). None of the attacks are successful under an appropriate multiplier of the median norm M-$r = 1.5$. For a looser bound (M-r: 15), the attack strategies are successful, and their relative performance diverges, with MP-PD achieving the highest malicious accuracy for **FMN** and MP-NT for **C10**.

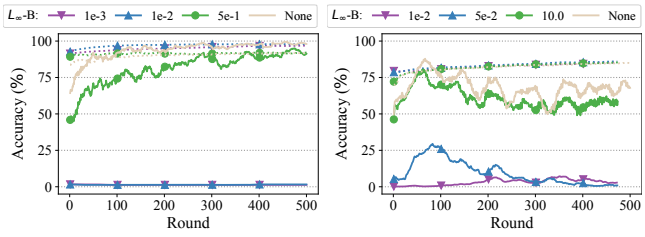
Growing number of attackers. We compare the performance of data poisoning with all three model poisoning in Fig. 18. All attacks show similar performance relative to the % of compromised clients per round, with MP-AT achieving the highest malicious accuracy improvement over data poisoning (at most 40%) as shown in Section §2.

L_∞-norm bound. we show that an L_∞-norm bound has the ability to defend against prototypical backdoor attacks, similar to the static and adaptive L_2-norm bounds shown in Section §2. An L_∞-norm bound of 0.01 for **FMN** and



(a) **FMN-P** (b) **C10-P**

Figure 18: Comparison of model poisoning attacks for various % of compromised clients per round. (M-r: 1.5)



(a) **FMN-P** (b) **C10-P**

Figure 19: Comparison of PGD attack under various static L_∞-norm bounds.

0.05 for **C10** is sufficient to successfully defend against a backdoor attack on prototypical samples, without impacting model convergence or accuracy (Fig. 19). Similar to the L_2-norm bound, when the norm bound is too tight, model convergence is impacted. Conversely, when the bound is too loose, the attacker is able to successfully inject the prototypical backdoor into the model.

Appendix B.

In this appendix, we discuss the security of the probabilistic checking optimization for L_∞-norm constraints in practical model poisoning attacks and justify our choice of security parameters.

The security of probabilistic checking is determined by the probability that a client is not detected (i.e., the failure probability) by the server while submitting a malicious update that violates the L_∞-norm constraint. Let $\mathbf{w}$ be a malicious update vector of length ℓ that violates the L_∞-norm constraint, i.e., $||\mathbf{w}||_\infty > B$, because $p_v \cdot \ell$ for $p_v \in (0, 1]$ parameters in $\mathbf{w}$ are above B. The server selects a fraction $p_c \in (0, 1]$ of the parameters $\mathbf{w}'$ in $\mathbf{w}$ uniformly at random and checks whether $||\mathbf{w}'||_\infty > B$. We can derive the probability of not detecting this malicious update vector by realizing this random selection by the server is equivalent to drawing $p_c \cdot \ell$ parameters without replacement. The probability of not detecting is then defined as the probability that all $p_c \cdot \ell$ draws are below-bound parameters for a pool in which $1-(p_v \cdot \ell)$ out of ℓ parameters are below the bound. This probability $\Pr(X = k)$ follows the hypergeometric distribution for k draws. Specifically, we

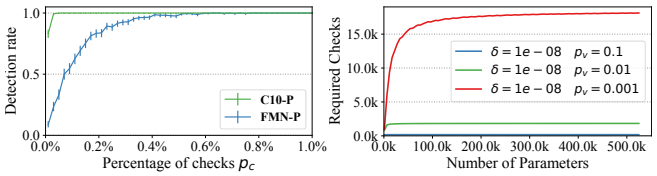
are interested in the case when $k = p_c \cdot \ell$, which defines the failure probability as

$$\Pr(X = p_c\ell) = Hyp(p_c\ell \mid \ell, \ell \cdot (1-p_v), p_c\ell)$$
$$= \frac{\binom{\ell(1-p_v)}{p_c \cdot \ell}\binom{\ell-\ell\cdot(1-p_v)}{p_c\ell - p_c\ell}}{\binom{\ell}{p_c\ell}}$$
$$= \frac{\binom{\ell(1-p_v)}{p_c\ell}}{\binom{\ell}{p_c\ell}}.$$

The security of the probabilistic checking approach depends on the assumption of the fractions p_v and p_c, where p_c can be set by the server. To configure p_c, RoFL assumes a fixed minimal p_v and a bounded failure probability of $1e-8$. In Fig. 20b, we present the number of parameters the server has to check for a fixed detection failure probability δ and fraction p_v of parameters assumed to be above the bound. We argue that in practical model poising attacks that depend on scaling, a significant fraction (i.e., $\geq p_v$) of parameters have to be above the bound for an attack to be successful. Here, we analyze the fractions of parameters above the bound for generic attacks from §2, and then show results for an adaptive attack in which the attacker optimizes for the number of scaled parameters.

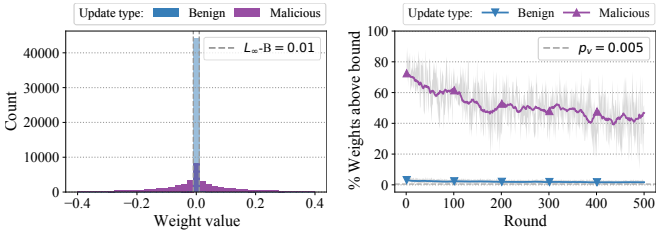
Generic Attacks. Fig. 21a shows the update weight distribution of a benign and a malicious update for the **FMN-P** single-shot attack. For the malicious update, 81.8% of parameters lie outside the L_∞-bound of 0.01, versus 0.87% for the benign update. In the continuous attack scenario, **FMN-P**, a similarly large fraction of parameters greater than 47% are above the bound in each round (Fig. 21b).

Adaptive attacks. In the adaptive case, we consider an adversary that optimizes for a minimal number of parameters above the bound in the poisoned update. The adaptive attacker optimizes their model poisoning update by selecting a fraction of K weights to scale based on their magnitude. We instantiate the following *top-K* adaptive attacker: Upon receiving the global model, the attacker crafts a malicious update as in the generic case, but scales only the $K \cdot \ell$ largest weights, clipping the remaining weights to the bound before sending the update to the server. Fig. 22 shows the performance of the top-K **C10-P** single-shot attack for varying percentages of K. Attacks are always successful for $K \geq 10\%$, which means the attacker does not have to scale the smallest 90% of weights to attack successfully. However, attack success starts to decrease for $K < 10\%$, with success below 50% for $K = 5\%$. Finally, the attack becomes completely ineffective (0% success) for $K \leq 2\%$. In **FMN-P**, the attacker has to scale fewer update parameters for success, but still a significant fraction of over 2 percent must be above the bound. In Fig. 20a, we show the effect of varying p_c on the detection rate for specific top-K attack updates in both **FMN-P** and **C10-P** tasks. The detection rate quickly converges to 1.0 as p_c increases. In our analysis, we opt for 0.005 as a conservative estimate for p_v, which is well below the observations in the generic and adaptive attacks.



(a) (b)

Figure 20: (a) The server only has to check a small fraction of parameters p_c to ensure a high detection rate on two malicious updates created using the *top-K* adaptive attack. (b) Number of update parameter ranges the server has to check with a failure probability of δ and the assumption that a fraction p_v of the parameters are above the bound.



(a) (b)

Figure 21: Generic attacks: (a) Weight distribution of a malicious update (**FMN**, **FMN-P**). Parameters outside of the interval $[-0.4, 0.4]$ are hidden. (b) Percentage of weights outside the L_∞-norm bound of 0.01 for malicious and benign clients per round (**FMN**, **FMN-P**). The total number of weights is 44426. We plot the assumption $p_v = 0.005$ as used in the evaluation.

Appendix C.

C.1. Analysis Setup

We first describe the details of the tasks in the analysis and then continue with the configurations of the attacks for these tasks.

Analysis Tasks. We consider two image classification tasks that have been used before in the context of FL backdoor attacks to provide a comparison to previous work. The first task (**FMN**) is a digit classification task on the Federated-MNIST dataset, with $n = 3383$ available clients and $m = 30$ clients selected per learning round, following [86, 90]. This dataset consists of samples from 3383 hand writers and is inherently non-IID because of the per-writer grouping, which simulates a natural distribution of training data. The second task (**C10**) is an image classification on the CIFAR-10 dataset. We divide the dataset among $n = 100$ clients and select $m = 40$ clients in each round. We distribute the images to clients in a non-IID fashion using a Dirichlet distribution with the parameter 0.9 in all dimensions. The full hyperparameters for the setups of **FMN** and **C10** are shown in Table 4. For **C10**, we use standard data augmentations (random horizontal flips and shifts) to reduce overfitting on the training dataset.

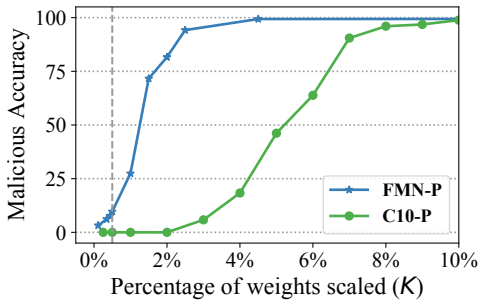


Figure 22: Top-K success attack on probabilistic checking.

The global learning rate η controls the fraction of the model that is updated each round. For **FMN**, the full model is updated with the average of $m = 30$ client updates ($\eta = \frac{3383}{30}$). For **C10**, we follow [7] and use a global learning rate of 1, indicating that 40% of the model is replaced every round. We pre-train the models for both tasks once and use the same weights as a starting point for each experiment, similar to [90]. The pre-trained models have not yet fully converged to show the effect of different configurations on model convergence. For **FMN**, we use a pre-trained LeNet5 with 88% test accuracy, and for **C10** we use a ResNet-20 with 80% test accuracy. The full configuration of the models is given in §C.2.1. All experiments in our analysis, except those in Figs. 3 and 8, show the accuracy as a moving average with a window size of 20 to improve readability.

Attack Tasks. For the **FMN** task, we consider a backdoor attack targeting prototypical inputs that classifies images containing the number 7 from 30 randomly selected handwriters as the number one instead (**FMN-P**, 310 images in total), following [86]. For a tail backdoor attack, we follow [90] and use images of sevens from the ARDIS dataset that are mostly in the European style (with a horizontal bar in the middle), unlike the sevens in F-MNIST (**FMN-T**, 660 images). On the **C10** task, we use a prototypical backdoor attack from [7]: classifying images of green cars as birds (**C10-P**, 30 images). For the tail backdoor attack, we use images from airplanes of Southwest airline that are collected by [90] to be introduced as tail backdoor images for CIFAR-10 (**C10-T**, 784 images).

Attack Success. For the **FMN-P** attack, attacker success is defined by the malicious classification accuracy on the backdoor images. For the **C10-P** attack, we measure accuracy using three backdoor samples that were not made available to the attacker during training. Test images are augmented into 200 versions per test image and used to evaluate the attacker's success in the global model [7]. For the tail attacks (**FMN-T**, **C10-T**), the images are already grouped into train and test sets, the latter of which are used to evaluate accuracy.

Attack Strategies. We discuss the configuration of the backdoor injection attacks used by the adversary to create a malicious model update. We explore two attacker models: data poisoning and model poisoning. In the data poisoning strategy, the attacker follows the training process (i.e., hyperparameters) as the benign clients but is allowed to insert a number of malicious samples into each batch according to the poison ratio. For model poisoning, we explore three different attacks that are adaptive with regard to the norm bound that is enforced by the server.

Client	FMN	C10
# Training samples	341873	50000
Model	LeNet5	ResNet-20
# Classes	10	10
# Total number of clients	3383	100
# Selected clients per round	30	40
# Local epochs per round	5	2
Batch size	32	64
Optimizer	SGD	
Learning rate	0.01	0.02

TABLE 4: Hyperparameters for honest clients.

The model poisoning clients train the model for a fixed amount of SGD steps, determined by the number of epochs and batches shown in Table 5. Each batch contains benign samples and a number of target samples mislabeled with the target class. The amount of malicious samples in each batch is controlled by the poison ratio. The optimal backdoor injection attack also includes benign samples because it allows the attacker to move the model into a direction where both objectives are satisfied simultaneously, making the attacker's update harder to reverse by honest client updates [7, 90]. We apply learning rate decay to malicious training to ensure the model update has good accuracy on both the main and backdoor tasks. The learning rate starts at 0.1 and decays step-wise with $\frac{1}{10}$ every $\frac{1}{3}$ of the total number of steps. We now discuss details specific to the attacks.

PGD When an L_p-norm bound B is present, the adversary uses Projected Gradient Descent (PGD) to adaptively craft malicious updates. Concretely, after every SGD step, the adversary projects its model update $\Delta\hat{\mathbf{w}}$ back to a point in the constraint set. For the norm bound, this set is defined as $||\Delta\hat{\mathbf{w}}||_p \leq \frac{B}{\gamma}$, where γ is the scaling factor. More precisely, similar to [90], during an epoch, we project the update back onto a slightly larger space ($1.2\frac{B}{\gamma}$). Only after the last SGD step of the epoch do we use γ as a scaling factor to satisfy the norm constraint (Table 5). In practice, this approach leads to a higher attacker success, similar as to what was reported in [90].

Neurotoxin We choose a mask ratio $k = 5$, which means that the malicious update is projected onto the bottom 95% of frequently updated weights using PGD. This value for the mask ratio achieved good overall performance in both the original attack paper and in our results.

Anticipate We follow the hyperparameters used in the original attack as closely as possible and set the number of anticipate steps $k = 9$ for **C10**. We choose $k = 5$ anticipate steps for **FMN** because the Federated-MNIST dataset was not used to evaluate the attack.

Setup	FMN	C10
# Local epochs per round	10	6
# Batches	25	10
Batch size	32	64
# Poison samples per batch	12	20
Optimizer	SGD	
Learning rate	0.1	0.1
Scale factor	30	100

TABLE 5: Hyperparameters for model poisoning attacks

C.2. RoFL Evaluation Setup

We now describe additional details of the experiments performed as part of the evaluation.

Microbenchmarks. For each reported result, we take the average of 4 measurements recorded after a single warm-up mock execution.

End-to-end. We evaluate the end-to-end performance of four different models. We use the two models that are also used in the analysis, and we evaluate two additional models. ResNet-20 [51] (`CIFAR-10 L`, 273k params) and LeNet5 [58] (`CIFAR-10 S`, 62k params). In addition, we evaluate the performance for a smaller CNN network (`MNIST`, 19k params) and a text-generative LSTM model (`Shakespeare`, 818k params). An overview of the tasks and models is shown in Table 2. The configuration of the end-to-end setup experiments is similar to those of the analysis, except for some minor changes. We increase the number of clients in the evaluation to 48 to benchmark the performance of a larger deployment. In each training round, all 48 clients are selected. For `MNIST`, we use a learning rate of 0.05; for `CIFAR-10 S`, a learning rate of 0.01; for `CIFAR-10 L`, a learning rate of 0.05; and for `Shakespeare`, 0.3. In RoFL, we apply 8-bit probabilistic quantization with 7 fractional bits for the floating point numbers in the client updates for encryption.

5. https://github.com/rstudio/keras/blob/master/vignettes/examples/cifar10_resnet.py

C.2.1. Model Architectures. We use the following model architectures for analysis and evaluation (end-to-end) experiments.

CNN. For the `MNIST` task, we use a standard convolutional neural network (CNN) architecture, consisting of two convolution layers of 8 and 4 filters, respectively, with kernel size 3-by-3. These are followed by a 2-by-2 max-pooling layer, a fully connected layer of 32 units and then the output layer. The model has 19k trainable parameters.

LeNet5. We make use of the standard LeNet5 [58] convolutional neural network for both **FMN** in the analysis and `CIFAR-10 S`. Note that the input layers for both applications of the model are different: 28-by-28, 1 channel for **FMN** and 32-by-32, 3 channels for `CIFAR-10 S`.

ResNet-20. We use the ResNet [51] model for experiments in the analysis (**FMN**) as well as in evaluation experiments (`CIFAR-10 L`). We use the ResNet implementation provided by the Keras ResNet CIFAR-10 example[5], corresponding to the ResNet20v1 model.

LSTM. We use the LSTM architecture as used in the LEAF federated learning benchmark [21] and the original federated learning paper [63]. The architecture consists of an embedding layer of size 8 followed by a two-layer stacked LSTM with 256 hidden units. The LSTMs are connected to a dense output layer with a width of 80, which corresponds to the vocabulary size.

Appendix D.

In Algorithm 1, we show how we extend the SecAgg protocol by Bell et al. [8] by highlighting the adaptations introduced by RoFL in *italics*.

Algorithm 1 Maliciously secure SecAgg [8] augmented with *input validation for L_∞-norm and L_2-norm* constraints.

Parties: Clients $1, 2, \ldots, n$ and server.
Public Parameters: Number of parameters ℓ, input domain $\mathbb{Z}_q^\ell$ for prime q, seed expansion function $\text{PRG} : \{0,1\}^\lambda \to \mathbb{Z}_q^\ell$, maximum fraction of dropout clients δ *, generators g and h of a cyclic group $\mathbb{G}$ and norm bound B.*
Input: $\mathbf{w}_i \in \mathbb{Z}_q^\ell$ by each client i.
Output: $\mathbf{z} \in \mathbb{Z}_q^\ell$ by the server.

Stage 1: Public-key commitments. (The server is assumed to behave semi-honestly in this stage.)

(1) Each client i generates keypairs $(\text{sk}_{i,1}, \text{pk}_{i,1})$, $(\text{sk}_{i,2}, \text{pk}_{i,2})$ and sends $(\text{pk}_{i,1}, \text{pk}_{i,2})$ to the server.

(2) The server commits to both vectors of public key $\text{pk}_1 = (\text{pk}_{i,1})_i$ and $\text{pk}_2 = (\text{pk}_{i,2})_i$ by means of a Merkle tree and sends the root hashes $h_{\text{root},1}$ and $h_{\text{root},2}$ to each client.

Stage 2: Distributed graph generation.

(3) Each client i selects k neighbors by sampling randomly and without replacement k times from the set of n clients, and sends the resulting set $\text{N}_\rightarrow(i)$ of outgoing neighbors to the server. The choices made by all clients implicitly define a set of "ingoing" neighbors for client i, denoted as $\text{N}_\leftarrow(i)$. Denote $\text{N}(i) = \text{N}_\rightarrow(i) \cup \text{N}_\leftarrow(i)$.

(4) The server sends $\text{N}_\leftarrow(i), (j, \text{pk}_{j,1}, \text{pk}_{j,2})_{j \in \text{N}(i)}$ to client i, together with two Merkle inclusion proofs for roots $h_{\text{root},1}$ and $h_{\text{root},2}$.

(5) Client i aborts if the server sent more than $3k + k$ public keys. Otherwise, the client verifies the Merkle inclusion proofs and that the public keys sent by the server correspond to the neighbors in $\text{N}_\rightarrow(i)$, and aborts otherwise.

Stage 3: Secret sharing.

(6) Each client i that has not dropped out:
- Generates a random PRG seed b_i.
- Computes two sets of shares $H_i^{\text{b}} = \{h_{i,1}^{\text{b}}, \ldots, h_{i,k}^{\text{b}}\} = \texttt{ShamirSS}(t, k, \text{b}_i)$ and $H_i^s = \{h_{i,1}^s, \ldots, h_{i,k}^s\} = \texttt{ShamirSS}(t, k, \text{sk}_{i,1})$.
- Sends to the server messages $m_j = (j, o_{i,j})$, where $o_{i,j} \leftarrow \text{E}_{\text{auth}}.\text{Enc}(k_{i,j}, i \,||\, j \,||\, h_{i,j}^{\text{seed}} \,||\, h_{i,j}^s)$ for each $j \in \text{N}_\rightarrow(i)$, where o_{ij} is a cipthertext encrypted under $k_{i,j} = \texttt{KA.Agree}(\text{sk}_{i,2}, \text{pk}_{j,2})$.

(7) The server aborts if it receives messages from less than $(1-\delta)n$ clients. Otherwise, it sends all messages $(j, o_{i,j})$ to client j. The set $A_j \subseteq \text{N}(j)$ contains the neighbors for which client j received such a message.

Stage 4: Masking and proving.

(8) Each client i that has not dropped:
- Computes a shared random seed $\text{b}_{i,j}$ as $\text{b}_{i,j} = \texttt{KA.Agree}(s_{ki,1}, pk_{j,1})$ with every $j \in A_i$
- Computes its masking key $\mathbf{r}_i$ by expanding the pairwise shared random seeds as $\mathbf{r}_i = \mathbf{s}_i + \sum_{j \in A_i, j<i} \mathbf{s}_{ij} - \sum_{j \in A_{1,i}, j<i} \mathbf{s}_{ij}$ for $\mathbf{s}_{ij} = \text{PRG}(\text{b}_{i,j})$ and $\mathbf{s}_i = \text{PRG}(\text{b}_i)$.
- *Computes ElGamal commitments* $\mathbf{c}_i = Enc(\mathbf{w}_i, \mathbf{r}_i) = Com_{EG}(\mathbf{w}_i, \mathbf{r}_i)$.
- *Generates compressed well-formedness proof* $\pi^{Enc(\mathbf{w}_i, \mathbf{r}_i)}$ *that* $\mathbf{c}_i = (g^{\mathbf{w}_i} h^{\mathbf{r}_i}, g^{\mathbf{r}_i})$.
- *Generates Bulletproof range proof for each parameter* $\pi^{\|\mathbf{w}_i\|_\infty < B}$.
- *Computes Pedersen commitments to the square of each parameter* $\mathbf{c}_i' = Com_{PD}((\mathbf{w}_i)^2, \mathbf{r}_i')$ *where* $\mathbf{r}_i' \in \mathbb{Z}_q^\ell$ *is generated using randomly sampled field elements.*
- *Generates squared well-formedness proofs for each parameter* π^{sq}.
- *Generates a Bulletproof range proof the sum of squares is bounded* $\pi^{\|\mathbf{w}_i\|_2^2 < B^2}$.
- Signs the message $m_{i,j} = (\text{"included"} \,||\, i \,||\, j)$ with $\text{sk}_{i,2}$ to obtain a signature $\sigma_{i,j}^{\text{incl}}$ for all $j \in A_j$.
- Sends to the server $(m_{i,j}, \sigma_{i,j}^{\text{incl}})_{j \in A_j}$ *and* $(\mathbf{c}_i, \pi^{\text{Enc}(\mathbf{w}_i, \mathbf{r}_i)}, \pi^{\|\mathbf{w}_i\|_\infty < B}, \mathbf{c}_i', \pi^{sq}, \pi^{\|\mathbf{w}_i\|_2^2 < B^2})$.

Stage 5: Unmasking.

(9) If the server receives fewer than $(1-\delta)n$ during a fixed time period, it aborts. Otherwise, it defines a global set of dropouts $\mathcal{D}$ and a set of survivors $\mathcal{S}$. It then sends the messages and signatures $(m_{j,i}, \sigma_{j,i}^{\text{incl}})$ to every client $i \in \mathcal{S}$, along with the sets $\mathcal{D}_i = \text{N}(i) \cap \mathcal{D}$ (its incoming neighbors that are dropouts) and $\mathcal{S}_i = \text{N}(i) \cap \mathcal{S}$ (its incoming neighbors that are not).

(10) Each client i that has not dropped out checks that $\mathcal{D}_i \cap \mathcal{S}_i = \emptyset$, that $\mathcal{S}_i, \mathcal{D}_i \subseteq \text{N}(i) \cap A_i$, and that each signature $\sigma_{j,i}^{\text{incl}}$ is valid for message $m_{j,i}$ for all $j \in \mathcal{S}_i$, and aborts otherwise.

(11) Each remaining client i, $\forall j \in \mathcal{S}_i$, signs a message $m_{i,j} = (\text{sk}_{i,2}, \text{"ack"} \,||\, i \,||\, j)$ using $\text{sk}_{i,2}$ and sends $(m_{i,j}, \sigma_{i,j}^{\text{ack}})_j$ to the server.

(12) The server aborts if it receives fewer than $(1-\delta)n$ responses *and verifies all proofs* $\pi^{Enc(\mathbf{w}_i, \mathbf{r}_i)}$, $\pi^{\|\mathbf{w}_i\|_\infty < B}$, $\pi^{sq}, \pi^{\|\mathbf{w}_i\|_2^2 < B^2}$, *aborting if any proofs fail to verify.* Otherwise, it forwards all messages $(j, m_{i,j}, \sigma_{i,j}^{\text{ack}})$ to client j.

(13) Each remaining client checks that all the signatures are valid using $\text{pk}_{j,2}$ and aborts if any of them fails to verify. Once client j receives p valid signatures from its neighbors, they send $\{i, h_{i,j}^{\text{b}}\}_{i \in \mathcal{D}_j}$ and $\{i, h_{i,j}^{\text{s}}\}_{i \in \mathcal{S}_j}$ to the server, which it has obtained by decrypting the ciphertexts $o_{i,j}$ received in Step 6.

(14) The server aborts if it receives fewer than $(1-\delta)n$ responses, and otherwise:
- Collects, for each client $i \in \mathcal{S}$, the set of all received shares in H_i^{b}. It aborts if there are fewer than t shares and otherwise recovers b_i and $\mathbf{s}_i$ using the t shares received from the lowest client IDs.
- Collects, for each client $i \in \mathcal{D}$, the set of all shares in H_i^s. It aborts if there are fewer than t shares and it recovers $\text{sk}_{i,1}$ and $\mathbf{s}_{ij}$ for all $j \in \text{N}(i)$ otherwise.
- Computes the decoding key $\mathbf{r} = \sum_{i \in \mathcal{S}} (\mathbf{s}_i + \sum_{j \in \mathcal{D}_i, j<i} \mathbf{s}_{ij} - \sum_{j \in \mathcal{D}_i, i<j} \mathbf{s}_{ij})$.
- Outputs $\sum_{i \in \mathcal{S}} \mathbf{w}_i$ as $\texttt{Dec}(\sum_{i \in \mathcal{S}} \mathbf{c}_i, \mathbf{r})$ by solving the discrete logarithm with respect to g.